

ĐẠI HỌC BÁCH KHOA HÀ NỘI

*

BÁO CÁO HỌC PHẦN

PROJECT 1



SOICT

TÊN ĐỀ TÀI

HỆ THỐNG QUẢN LÝ CÔNG VIỆC

Giảng viên hướng dẫn : Th.S Phạm Quang Hiếu
Sinh viên thực hiện : Nguyễn Trọng Nhân – 202490073
Lớp : CNTT 1.1 - K69

Hà Nội 05 - 2026

Mục lục

Lời cảm ơn	5
1 CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI	6
1.1 Giới thiệu đề tài	6
1.2 Tính cấp thiết của đề tài	6
1.3 Mục tiêu đề tài	7
1.4 Phạm vi đề tài	7
1.5 Đối tượng sử dụng	8
2 CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	10
2.1 Cách tiếp cận và hướng giải quyết	10
2.1.1 Phương pháp phân tích bài toán	10
2.1.2 Hướng thiết kế giải pháp	10
2.1.3 Các quyết định kỹ thuật chính	11
2.2 Các nền tảng và công nghệ sử dụng	11
2.2.1 Các nền tảng liên quan đến ứng dụng	11
2.2.2 Các nền tảng liên quan đến dịch vụ đám mây	14
2.2.3 Các nền tảng khác	17
3 CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	20
3.1 Phân tích yêu cầu	20
3.1.1 Yêu cầu chức năng	20
3.1.2 Yêu cầu phi chức năng	23
3.2 Phân rã chức năng	23
3.2.1 Xác thực và quản lý tài khoản	23
3.2.2 Quản lý Workspace	27
3.2.3 Quản lý Board	29
3.2.4 Quản lý List	32
3.2.5 Quản lý Card	33
3.2.6 Kéo thả (Drag and Drop)	36
3.2.7 Quản lý Checklist	37
3.3 Phân tích dữ liệu	38
3.3.1 Các thực thể chính	38
3.3.2 Quan hệ giữa các thực thể	39
3.4 Thiết kế cơ sở dữ liệu	40
3.4.1 Kiểu liệt kê	40
3.4.2 Bảng tra cứu	41
3.4.3 Bảng nghiệp vụ	42
3.5 Thiết kế tổng thể hệ thống	48

4	CHƯƠNG 4: TRIỂN KHAI VÀ KẾT QUẢ	51
4.1	Các chức năng đã hoàn thành	51
4.2	Kết quả đạt được	52
4.2.1	Thống kê tổng quát	52
4.2.2	Kết quả kiểm thử theo module	52
4.2.3	Các màn hình giao diện đã hoàn thành	54
4.3	Hình ảnh minh họa	58
4.4	Đánh giá kết quả	59
4.4.1	Ưu điểm	59
4.4.2	Hạn chế	59
5	CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	61
5.1	Kết luận	61
5.1.1	Kết quả đạt được	61
5.2	Các vấn đề còn tồn tại	62
5.2.1	Chức năng chưa hoàn thiện	62
5.2.2	Hạn chế kỹ thuật	62
5.3	Phương hướng phát triển	63
5.3.1	Ưu tiên cao — Hoàn thiện hệ thống hiện tại	63
5.3.2	Ưu tiên trung bình — Mở rộng tính năng	64
5.3.3	Ưu tiên thấp — Phát triển dài hạn	64
A	PHỤ LỤC A: DANH SÁCH API ENDPOINT ĐẦY ĐỦ	66
B	PHỤ LỤC B: CHI TIẾT BẢNG CƠ SỞ DỮ LIỆU	68
C	PHỤ LỤC C: DANH SÁCH KỊCH BẢN KIỂM THỬ	74

Danh sách hình vẽ

1	Logo ReactJS	12
2	Logo Java Spring Boot	13
3	Logo PostgreSQL	14
4	Logo Docker	18
5	Logo Nginx	18
6	Thiết kế cơ sở dữ liệu	40
7	Kiến trúc tổng thể của hệ thống Nello	50
8	Màn hình Đăng ký tài khoản	55
9	Màn hình Xác nhận OTP	55
10	Màn hình Đăng nhập	56
11	Màn hình Dashboard	56
12	Màn hình Chi tiết Workspace	57
13	Màn hình Board Kanban	57
14	Tính năng kéo thả Card giữa các List	58
15	Modal Chi tiết Card với Checklist, Due Date và thành viên . .	58

Danh sách bảng

1	Yêu cầu chức năng nhóm 1	20
2	Yêu cầu chức năng nhóm 2	21
3	Yêu cầu chức năng nhóm 3	21
4	Yêu cầu chức năng nhóm 4	21
5	Yêu cầu chức năng nhóm 5	22
6	Yêu cầu chức năng nhóm 6	22
7	Yêu cầu chức năng nhóm 7	22
8	Kiểu liệt kê <code>role_name_enum</code>	40
9	Bảng <code>roles</code>	41
10	Bảng <code>images</code>	41
11	Bảng <code>workspace_categories</code>	41
12	Bảng <code>users</code>	42
13	Bảng <code>workspaces</code>	43
14	Bảng <code>workspace_members</code> – quan hệ nhiều-nhiều giữa <code>users</code> và <code>workspaces</code>	43
15	Bảng <code>boards</code>	44
16	Bảng <code>board_members</code> – quan hệ nhiều-nhiều giữa <code>users</code> và <code>boards</code>	44
17	Bảng <code>lists</code>	45
18	Bảng <code>cards</code>	46
19	Bảng <code>card_members</code> – quan hệ nhiều-nhiều giữa <code>users</code> và <code>cards</code>	46
20	Bảng <code>checklists</code>	47
21	Bảng <code>checklist_items</code>	48
22	Danh sách chức năng đã hoàn thành (27/31 — 87,1%)	52
23	Tổng hợp kết quả thực hiện	52
24	Kết quả kiểm thử module Authentication	53
25	Kết quả kiểm thử module Workspace & Board	53
26	Kết quả kiểm thử module Card & Checklist	53
27	Kết quả kiểm thử tích hợp Frontend (W6)	54
28	Kết quả kiểm thử giao diện Board (W7)	54
29	Tổng hợp kết quả học phần Project I	61
30	Các tính năng chưa được triển khai	62
31	Danh sách API endpoint đầy đủ của hệ thống	67

Lời cảm ơn

Em xin chân thành cảm ơn giảng viên hướng dẫn, Th.S Phạm Quang Hiếu, đã tận tình hướng dẫn và hỗ trợ em trong suốt quá trình thực hiện học phần Project 1.

Em cũng xin cảm ơn Trường Công nghệ Thông tin và Truyền thông, Đại học Bách Khoa Hà Nội đã tạo điều kiện học tập và nghiên cứu thuận lợi.

Trong quá trình thực hiện báo cáo, do kiến thức và kinh nghiệm còn hạn chế, không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý từ thầy cô để có thể hoàn thiện hơn trong thời gian tới.

Sinh viên thực hiện

Nguyễn Trọng Nhân

1 CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1 Giới thiệu đề tài

Trong quá trình quản lý dự án, việc tổ chức và theo dõi công việc luôn là một bài toán phức tạp đối với các cá nhân hay tổ chức. Khi số lượng công việc gia tăng, các phương pháp quản lý truyền thống hoặc rời rạc thường bộc lộ nhiều hạn chế, dẫn đến những khó khăn trực tiếp trong việc kiểm soát tiến độ, phân chia nguồn lực và duy trì sự phối hợp giữa các thành viên. Tình trạng thất lạc thông tin, thiếu minh bạch về trạng thái thực hiện các hạng mục công việc làm suy giảm đáng kể hiệu suất chung. Xuất phát từ thực trạng này, bài toán đặt ra là cần xây dựng một giải pháp phần mềm hỗ trợ quản lý dự án tập trung, trực quan, giúp người dùng dễ dàng tổ chức các đầu việc, theo dõi tiến độ và nâng cao hiệu quả cộng tác.

1.2 Tính cấp thiết của đề tài

Hiện nay, mặc dù đã có nhiều giải pháp phần mềm hỗ trợ quản lý, nhiều cá nhân và tổ chức tại Việt Nam vẫn sử dụng các phương pháp thủ công như ghi chép sổ tay, bảng trắng, hoặc các công cụ kỹ thuật số rời rạc và thiếu tính trực quan như email, bảng tính Excel, hay các ứng dụng nhắn tin nhanh. Các phương thức tiếp cận truyền thống này bộc lộ nhiều hạn chế nghiêm trọng trong môi trường làm việc hiện đại:

- **Thất lạc và phân mảnh thông tin:** Các yêu cầu công việc, phản hồi và tài liệu đính kèm bị phân tán trên nhiều kênh giao tiếp khác nhau, dẫn đến khó khăn trong việc tìm kiếm và dễ bỏ sót nhiệm vụ.
- **Khó theo dõi tiến độ tổng thể:** Người quản lý và các thành viên gặp nhiều khó khăn để có được cái nhìn tổng quan về trạng thái hiện tại của dự án, dẫn đến chậm trễ tiến độ mà không có biện pháp cảnh báo hoặc xử lý kịp thời.
- **Thiếu tính tương tác trực quan:** Các công cụ truyền thống không hỗ trợ cập nhật động trạng thái công việc và luồng di chuyển của các nhiệm vụ theo thời gian thực.
- **Giảm hiệu suất cộng tác nhóm:** Việc phân chia trách nhiệm và giao việc không được định nghĩa rõ ràng, gây chồng chéo công việc và suy giảm hiệu quả làm việc chung.

Từ những hạn chế trên, việc nghiên cứu và xây dựng “Hệ thống quản lý công việc” là vô cùng cấp thiết. Hệ thống được phát triển nhằm khắc phục các

nhược điểm của phương pháp truyền thống bằng việc cung cấp một giao diện quản lý trực quan theo mô hình Kanban. Giải pháp này giúp chuẩn hóa quy trình giao việc, nâng cao tính minh bạch trong tập thể, tối ưu hóa quy trình làm việc và giảm thiểu tối đa các sai sót thông tin trong suốt quá trình vận hành dự án.

1.3 Mục tiêu đề tài

Mục tiêu chính của đề tài là nghiên cứu, thiết kế và phát triển một nền tảng quản lý công việc và thẻ công việc (task/card) hiệu quả, trực quan, dễ tiếp cận và có khả năng tương tác cao. Các mục tiêu chi tiết bao gồm:

- **Xây dựng nền tảng quản lý khoa học:** Cho phép người dùng dễ dàng tổ chức công việc theo các không gian làm việc (workspace) riêng biệt và các bảng công việc (board) theo từng dự án cụ thể.
- **Quản lý thẻ công việc chi tiết:** Cung cấp đầy đủ các thuộc tính quản lý cho một nhiệm vụ như tiêu đề, mô tả chi tiết, gán thành viên thực hiện, thiết lập thời hạn hoàn thành (due date) và xây dựng các danh sách kiểm tra công việc con (checklist) để theo dõi chi tiết.
- **Tích hợp cơ chế tương tác trực quan:** Triển khai thành công tính năng kéo thả (drag and drop) các thẻ công việc giữa các danh sách trạng thái khác nhau, mang lại trải nghiệm mượt mà và trực quan cho người dùng.
- **Đồng bộ dữ liệu và cộng tác hiệu quả:** Đảm bảo các cập nhật về trạng thái công việc, phân công nhân sự và thời hạn được đồng bộ hóa nhanh chóng đến toàn bộ thành viên trong bảng công việc.

1.4 Phạm vi đề tài

Đề tài tập trung nghiên cứu và triển khai các chức năng cốt lõi phục vụ nhu cầu quản lý công việc hàng ngày của cá nhân và đội ngũ, cụ thể bao gồm:

- **Hệ thống xác thực và quản lý tài khoản:** Triển khai các tính năng đăng ký, đăng nhập, đăng xuất và cập nhật thông tin cá nhân của người dùng, tích hợp giải pháp xác thực bảo mật thông qua dịch vụ điện toán đám mây Amazon Cognito.
- **Quản lý Workspace:** Tạo lập, cập nhật thông tin, xóa và tổ chức các không gian làm việc để quản lý tập trung nhiều dự án liên quan.

- **Quản lý Board:** Tạo mới, cập nhật thông tin, xóa các bảng công việc theo dự án; triển khai tính năng tìm kiếm và thêm thành viên vào bảng công việc.
- **Quản lý danh sách công việc (List):** Tạo mới, đổi tên, thay đổi thứ tự sắp xếp và xóa các danh sách trong mỗi bảng nhằm phân loại trạng thái công việc (ví dụ: Cần làm, Đang làm, Đã hoàn thành).
- **Quản lý thẻ công việc (Card):** Thực hiện các thao tác CRUD (Tạo, Đọc, Cập nhật, Xóa) đối với thẻ công việc; hỗ trợ thêm mô tả chi tiết, gán thành viên chịu trách nhiệm, thiết lập thời hạn hoàn thành và hiển thị trạng thái cảnh báo trực quan khi công việc sắp đến hạn hoặc quá hạn.
- **Quản lý danh sách kiểm tra (Checklist):** Cho phép tạo các checklist trong mỗi card, thêm các hạng mục công việc con (checklist item), đánh dấu hoàn thành/chưa hoàn thành và hiển thị thanh tiến độ (progress bar) trực quan theo tỷ lệ phần trăm hoàn thành.
- **Tương tác kéo thả (Drag and Drop):** Phát triển thuật toán và giao diện hỗ trợ kéo thả thẻ công việc giữa các danh sách hoặc thay đổi vị trí của các thẻ trong cùng một danh sách, đảm bảo tính nhất quán về vị trí hiển thị giữa Frontend và Backend.

Các nội dung và tính năng nằm ngoài phạm vi nghiên cứu thực hiện của học phần bao gồm:

- Các chức năng phân tích số liệu nâng cao, dự báo tiến độ và tự động xuất các báo cáo thống kê định kỳ dưới dạng biểu đồ phức tạp.
- Việc tích hợp sâu hệ thống quản lý công việc với các nền tảng và dịch vụ của bên thứ ba như Slack, Google Calendar, hay các công cụ lưu trữ đám mây bên ngoài.
- Tối ưu hóa hạ tầng và cơ chế cân bằng tải để phục vụ cho hệ thống quy mô lớn có hàng triệu người dùng hoạt động đồng thời.

1.5 Đối tượng sử dụng

Hệ thống quản lý công việc hướng đến việc giải quyết nhu cầu của các nhóm đối tượng người dùng chính sau:

- **Cá nhân:** Người dùng có nhu cầu sắp xếp, lên kế hoạch và quản lý các công việc học tập, nghiên cứu hoặc làm việc cá nhân hàng ngày một cách khoa học, giúp tăng tính kỷ luật và hiệu suất cá nhân.

- **Nhóm làm việc và phòng ban:** Các đội ngũ phát triển dự án, nhóm nghiên cứu hoặc các phòng ban chức năng trong tổ chức cần một không gian cộng tác số thống nhất để phân công công việc, phối hợp thực hiện và cùng theo dõi tiến độ công việc một cách minh bạch, rõ ràng.
- **Quản lý dự án:** Những người chịu trách nhiệm điều phối và giám sát toàn bộ hoạt động của dự án, cần theo dõi tổng quan tiến độ của từng hạng mục công việc, kiểm soát thời hạn hoàn thành (deadlines) và tối ưu hóa việc phân bổ nguồn lực nhân sự.

2 CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Cách tiếp cận và hướng giải quyết

2.1.1 Phương pháp phân tích bài toán

Bài toán được tiếp cận theo hướng phân tích từ trên xuống (top-down), bắt đầu từ việc nghiên cứu các hệ thống quản lý công việc phổ biến hiện có trên thị trường như Trello và Jira, từ đó xác định mô hình nghiệp vụ, các thực thể dữ liệu cốt lõi và luồng hoạt động của hệ thống. Qua nghiên cứu, mô hình Kanban được lựa chọn làm nền tảng tổ chức công việc do tính trực quan và khả năng phản ánh trạng thái công việc một cách rõ ràng theo các cột trạng thái (To Do, In Progress, Done).

Mô hình dữ liệu nghiệp vụ được xác định xoay quanh sáu thực thể chính theo cấu trúc phân cấp:

- **Workspace:** không gian làm việc tổng thể của người dùng, quản lý tập trung nhiều dự án liên quan.
- **Board:** đại diện cho một dự án hoặc bảng công việc cụ thể bên trong Workspace.
- **List:** các cột trạng thái bên trong Board, thể hiện giai đoạn thực hiện của công việc.
- **Card:** nhiệm vụ cụ thể cần thực hiện, được đặt trong các List tương ứng.
- **Checklist:** danh sách các công việc con nằm bên trong thẻ công việc (Card), hỗ trợ theo dõi chi tiết tiến trình thực hiện.
- **ChecklistItem:** các hạng mục cụ thể bên trong Checklist, có thể được đánh dấu trạng thái hoàn thành.

2.1.2 Hướng thiết kế giải pháp

Giải pháp được thiết kế theo kiến trúc client-server với sự tách biệt rõ ràng giữa Frontend và Backend thông qua giao tiếp RESTful API. Hướng tiếp cận này giúp hai thành phần có thể phát triển song song, dễ dàng bảo trì và mở rộng độc lập. Toàn bộ logic nghiệp vụ được tập trung xử lý tại Backend, trong khi Frontend chịu trách nhiệm trình bày giao diện và tương tác với người dùng.

2.1.3 Các quyết định kỹ thuật chính

Các quyết định kỹ thuật quan trọng được đưa ra trong quá trình thiết kế hệ thống bao gồm:

- **Phân tách Frontend và Backend:** Frontend xây dựng bằng ReactJS, Backend xây dựng bằng Java Spring Boot, giao tiếp qua REST API, tạo ra sự linh hoạt cao trong phát triển và triển khai.
- **Ủy quyền xác thực cho dịch vụ bên thứ ba:** Sử dụng Amazon Cognito để quản lý toàn bộ vòng đời người dùng (đăng ký, xác thực email, đăng nhập), giúp Backend tập trung vào logic nghiệp vụ thay vì tự xử lý xác thực.
- **Lưu trữ token bằng HttpOnly Cookie:** Access token và refresh token được lưu trong HttpOnly cookie thay vì localStorage, ngăn chặn các cuộc tấn công XSS (Cross-Site Scripting).
- **Cơ sở dữ liệu quan hệ PostgreSQL:** Phù hợp với mô hình dữ liệu có cấu trúc quan hệ rõ ràng và hỗ trợ tốt cho JPA/Hibernate trong Spring Boot.
- **Triển khai trên nền tảng Cloud:** Ứng dụng được triển khai hoàn toàn lên môi trường điện toán đám mây thông qua hệ sinh thái các dịch vụ của AWS (bao gồm EC2, RDS, S3, CloudFront và ALB), giúp hệ thống tận dụng tối đa khả năng mở rộng linh hoạt, tối ưu chi phí hạ tầng và đảm bảo tính sẵn sàng cao.

2.2 Các nền tảng và công nghệ sử dụng

2.2.1 Các nền tảng liên quan đến ứng dụng

2.2.1.1 ReactJS

- **Giới thiệu:** ReactJS là thư viện JavaScript mã nguồn mở được phát triển bởi Meta (Facebook), chuyên dùng để xây dựng giao diện người dùng (UI) tương tác và có hiệu suất cao.



Hình 1: Logo ReactJS

- **Chức năng:** ReactJS hoạt động theo cơ chế component-based, cho phép chia nhỏ giao diện thành các thành phần độc lập, có thể tái sử dụng. Virtual DOM giúp cập nhật giao diện một cách hiệu quả bằng cách chỉ render lại những phần thực sự thay đổi, thay vì cập nhật toàn bộ cây DOM.
- **Vai trò trong hệ thống:** ReactJS được sử dụng để xây dựng toàn bộ tầng giao diện (View layer) của hệ thống quản lý công việc, bao gồm các màn hình xác thực (đăng ký, đăng nhập), màn hình Dashboard hiển thị danh sách Workspace và Board, cũng như màn hình Board chi tiết với các chức năng quản lý List, Card và Drag and Drop.
- **Lý do lựa chọn:** Đối với các dạng ứng dụng web trang đơn (Single Page Application - SPA), ReactJS là một thư viện phổ biến, được sử dụng rất nhiều bởi các lập trình viên nhờ tính linh hoạt và khả năng triển khai dễ dàng, nhanh chóng.

2.2.1.2 Java Spring Boot

- **Giới thiệu:** Spring Boot là framework phát triển ứng dụng Java được xây dựng trên nền tảng Spring Framework, cung cấp cơ chế tự động cấu hình (auto-configuration) giúp giảm thiểu thời gian thiết lập dự án và tập trung vào triển khai logic nghiệp vụ.



Hình 2: Logo Java Spring Boot

- **Chức năng:** Spring Boot hỗ trợ xây dựng các ứng dụng RESTful API thông qua module Spring Web, quản lý kết nối cơ sở dữ liệu thông qua Spring Data JPA và Hibernate, đồng thời cung cấp cơ chế dependency injection giúp tổ chức code theo hướng lập trình hướng đối tượng rõ ràng.
- **Vai trò trong hệ thống:** Spring Boot đóng vai trò là tầng Backend của hệ thống, xử lý toàn bộ logic nghiệp vụ, xây dựng và cung cấp các RESTful API cho Frontend tiêu thụ. Cấu trúc project được tổ chức theo mô hình phân lớp bao gồm: `controller`, `service`, `repository` và `config`.
- **Lý do lựa chọn:** Spring Boot là một framework hỗ trợ công việc viết code cho các lập trình viên một cách mạnh mẽ và tối ưu. Kết hợp cùng với Java - một ngôn ngữ lập trình chặt chẽ, có tính ổn định cao, việc sử dụng Spring Boot giúp giảm thiểu tối đa các lỗi hệ thống có thể xảy ra trong quá trình vận hành.

2.2.1.3 PostgreSQL

- **Giới thiệu:** PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở mạnh mẽ, hỗ trợ đầy đủ các tính năng của SQL chuẩn cùng nhiều tính năng nâng cao như kiểu dữ liệu JSON, UUID, các ràng buộc phức tạp.



Hình 3: Logo PostgreSQL

- **Chức năng:** PostgreSQL đảm bảo tính toàn vẹn dữ liệu thông qua các ràng buộc FK (foreign key), hỗ trợ các truy vấn phức tạp với nhiều bảng quan hệ, đồng thời cung cấp hiệu suất đọc/ghi ổn định phù hợp với các ứng dụng web thông thường.
- **Vai trò trong hệ thống:** PostgreSQL được sử dụng làm hệ quản trị cơ sở dữ liệu chính của hệ thống, lưu trữ toàn bộ dữ liệu nghiệp vụ bao gồm thông tin người dùng, Workspace, Board, List, Card, Checklist và các bảng quan hệ trung gian. Kết nối với Backend thông qua Spring Data JPA và trình điều khiển PostgreSQL JDBC.

2.2.1.4 Mô hình MVC (Model – View – Controller)

MVC là mẫu kiến trúc phần mềm (architectural pattern) phân tách ứng dụng thành ba thành phần độc lập: **Model** chịu trách nhiệm quản lý dữ liệu và logic nghiệp vụ; **View** đảm nhận việc trình bày dữ liệu ra giao diện người dùng; **Controller** đóng vai trò trung gian, tiếp nhận yêu cầu từ người dùng, điều phối Model xử lý và trả kết quả về View.

2.2.2 Các nền tảng liên quan đến dịch vụ đám mây

2.2.2.1 Amazon Cognito

- **Giới thiệu:** Amazon Cognito là dịch vụ quản lý danh tính và xác thực người dùng do Amazon Web Services (AWS) cung cấp, hỗ trợ các luồng xác thực theo chuẩn OAuth 2.0 và OpenID Connect.
- **Chức năng:** Amazon Cognito cung cấp hai thành phần chính: User Pool để quản lý người dùng và xác thực danh tính, và Identity Pool để cấp quyền truy cập tài nguyên AWS. Dịch vụ này tự động xử lý việc gửi email xác nhận, quản lý mật khẩu, làm mới token và bảo vệ tài khoản theo các chính sách bảo mật chuẩn.

- **Vai trò trong hệ thống:** Amazon Cognito được tích hợp trực tiếp với tầng Backend thông qua AWS SDK for Java để xử lý toàn bộ luồng xác thực người dùng. Theo kiến trúc này, Client không gọi trực tiếp đến Cognito mà tương tác qua Backend; Backend sẽ giao tiếp với Cognito để đăng nhập, đăng ký và nhận JWT (access token, refresh token), sau đó trả về cho Client.

2.2.2.2 Amazon RDS

- **Giới thiệu:** Amazon RDS (Relational Database Service) là dịch vụ cơ sở dữ liệu quan hệ được quản lý hoàn toàn bởi AWS, hỗ trợ nhiều hệ quản trị cơ sở dữ liệu phổ biến trong đó có PostgreSQL.
- **Chức năng:** Amazon RDS tự động hóa các tác vụ quản trị cơ sở dữ liệu như sao lưu định kỳ, vá lỗi phần mềm, phát hiện và khôi phục lỗi tự động (automatic failure detection and recovery) và mở rộng tài nguyên. Dịch vụ cũng cung cấp tính năng High Availability thông qua triển khai Multi-AZ, đảm bảo cơ sở dữ liệu luôn sẵn sàng phục vụ.
- **Vai trò trong hệ thống:** Amazon RDS được dự kiến sử dụng để hosting cơ sở dữ liệu PostgreSQL trên môi trường đám mây, thay thế cho cơ sở dữ liệu cục bộ trong quá trình phát triển, nhằm chuẩn bị cho việc triển khai hệ thống lên môi trường Staging trong giai đoạn tiếp theo.

2.2.2.3 Amazon EC2 (Elastic Compute Cloud)

- **Giới thiệu:** Amazon EC2 là một dịch vụ web cốt lõi của AWS, cung cấp khả năng điện toán (compute capacity) an toàn và có thể thay đổi kích thước linh hoạt trên đám mây.
- **Chức năng:** EC2 cho phép khởi tạo các máy chủ ảo (instances) để chạy các ứng dụng. Người dùng có toàn quyền kiểm soát cấu hình phần cứng ảo (CPU, RAM), hệ điều hành và cài đặt các phần mềm cần thiết trên đó.
- **Vai trò trong hệ thống:** EC2 được sử dụng làm máy chủ (đặt trong mạng nội bộ - Private Subnet) để triển khai cả ứng dụng Backend (chạy Spring Boot) và Frontend (chạy ReactJS qua Nginx và Docker). Việc đặt EC2 trong mạng private giúp cách ly hoàn toàn các máy chủ khỏi Internet, tăng cường tối đa tính bảo mật.

2.2.2.4 Amazon ALB (Application Load Balancer)

- **Giới thiệu:** Amazon ALB là một dịch vụ cân bằng tải ở tầng ứng dụng (Layer 7) của AWS, chuyên phân phối lưu lượng truy cập HTTP và HTTPS một cách thông minh.
- **Chức năng:** ALB có khả năng phân phối lưu lượng truy cập mạng từ bên ngoài đến nhiều máy chủ đích (như các EC2 instances). Nó cung cấp các tính năng như định tuyến linh hoạt, hỗ trợ bảo mật SSL/TLS và tự động kiểm tra sức khỏe (health check) của các máy chủ đích để loại bỏ các máy chủ đang gặp lỗi.
- **Vai trò trong hệ thống:** ALB đóng vai trò là điểm truy cập (entry point) duy nhất cho toàn bộ hệ thống, tiếp nhận các request từ người dùng và phân phối tải đều đến các máy chủ EC2. Điều này giúp nâng cao tính sẵn sàng, cải thiện hiệu suất và hỗ trợ mở rộng hệ thống một cách dễ dàng khi lưu lượng truy cập tăng cao.

2.2.2.5 Amazon S3 (Simple Storage Service)

- **Giới thiệu:** Amazon S3 là dịch vụ lưu trữ đối tượng (object storage) do AWS cung cấp, mang lại khả năng lưu trữ và truy xuất lượng dữ liệu bất kỳ từ mọi nơi trên Internet với độ bền và tính bảo mật cao.
- **Chức năng:** S3 cho phép lưu trữ không giới hạn các loại tệp tin tĩnh (static files) và dữ liệu phi cấu trúc. Nó cũng hỗ trợ tính năng lưu trữ website tĩnh (static website hosting) để phục vụ trực tiếp các tệp HTML, CSS, JavaScript cho các ứng dụng web.
- **Vai trò trong hệ thống:** Trong dự án này, do Frontend đã được đưa lên máy chủ EC2, Amazon S3 được sử dụng chuyên biệt làm nơi lưu trữ an toàn cho các tài nguyên đa phương tiện do người dùng tải lên, ví dụ như hình ảnh đại diện (avatar) hoặc các tệp đính kèm trong thẻ công việc (Card).

2.2.2.6 Amazon CloudFront

- **Giới thiệu:** Amazon CloudFront là dịch vụ mạng phân phối nội dung (Content Delivery Network - CDN) tốc độ cao của AWS, giúp truyền tải dữ liệu và ứng dụng tới người dùng trên toàn cầu với độ trễ cực thấp.

- **Chức năng:** CloudFront lưu trữ bộ nhớ đệm (caching) các nội dung tĩnh và động tại các vị trí biên (edge locations) trên khắp thế giới, giúp rút ngắn khoảng cách vật lý giữa người dùng và dữ liệu, từ đó cải thiện đáng kể tốc độ tải trang web.
- **Vai trò trong hệ thống:** CloudFront được cấu hình để chỉ tương tác trực tiếp với Client và Amazon S3. Nó đóng vai trò trung gian phân phối các tệp tin đa phương tiện (từ S3) đến tay người dùng một cách tối ưu nhất thông qua bộ nhớ đệm (caching), đồng thời cung cấp giao thức HTTPS bảo mật và hạn chế truy cập công khai vào S3.

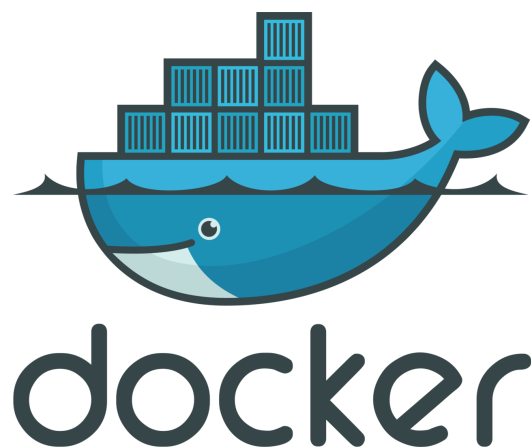
2.2.2.7 JWT (JSON Web Token)

- **Giới thiệu:** JWT là tiêu chuẩn mở (RFC 7519) định nghĩa cách truyền tải thông tin an toàn giữa các bên dưới dạng một đối tượng JSON được ký số (digitally signed).
- **Chức năng:** Một JWT bao gồm ba phần được mã hóa Base64 và ngăn cách bởi dấu chấm: Header (loại thuật toán ký), Payload (thông tin người dùng và thời hạn token) và Signature (chữ ký xác thực). Token có thể được xác minh mà không cần truy vấn cơ sở dữ liệu, giúp giảm tải cho hệ thống.
- **Vai trò trong hệ thống:** JWT được Amazon Cognito phát hành và trả về cho Backend sau khi người dùng đăng nhập thành công. Backend sau đó trả JWT về cho Client để lưu trữ trong HttpOnly cookie. Backend sử dụng token này để xác thực chữ ký trên mỗi request gọi API tiếp theo nhằm đảm bảo tính an toàn.

2.2.3 Các nền tảng khác

2.2.3.1 Docker

- **Giới thiệu:** Docker là nền tảng container hóa mã nguồn mở cho phép đóng gói ứng dụng cùng toàn bộ các phụ thuộc (thư viện, cấu hình, runtime) vào một đơn vị độc lập gọi là container. Container Docker có thể chạy nhất quán trên mọi môi trường từ máy phát triển đến máy chủ sản xuất.



Hình 4: Logo Docker

- **Chức năng:** Docker cung cấp cơ chế cô lập tiến trình thông qua namespace và cgroup của nhân Linux, giúp nhiều container chạy song song trên cùng một máy chủ mà không xung đột tài nguyên. Dockerfile cho phép định nghĩa môi trường build theo dạng khai báo, đảm bảo tính tái tạo (reproducibility) của quá trình build.
- **Vai trò trong hệ thống:** Ứng dụng Frontend (ReactJS) được đóng gói thành Docker image và triển khai trên máy chủ EC2. Docker đảm bảo môi trường chạy ứng dụng nhất quán giữa môi trường phát triển cục bộ và môi trường triển khai trên AWS, đồng thời đơn giản hóa quy trình cập nhật phiên bản ứng dụng.

2.2.3.2 Nginx

- **Giới thiệu:** Nginx là web server hiệu năng cao và reverse proxy mã nguồn mở, nổi tiếng với khả năng xử lý đồng thời lượng lớn kết nối với mức tiêu thụ tài nguyên thấp nhờ kiến trúc hướng sự kiện bất đồng bộ (event-driven, asynchronous).



Hình 5: Logo Nginx

- **Chức năng:** Nginx đảm nhiệm các vai trò phục vụ tệp tĩnh (static file serving), reverse proxy chuyển tiếp request đến ứng dụng phía sau, cấu hình HTTPS và gzip compression để giảm kích thước phản hồi. Ngoài ra, Nginx hỗ trợ cấu hình định tuyến linh hoạt thông qua file cấu hình theo cú pháp khai báo.
- **Vai trò trong hệ thống:** Nginx được cài đặt bên trong Docker container trên máy chủ EC2 Frontend, đóng vai trò là web server phục vụ các tệp tĩnh của ứng dụng ReactJS (HTML, CSS, JavaScript). Ngoài ra, Nginx xử lý định tuyến phía client (client-side routing) bằng cách chuyển hướng mọi request về file `index.html`, đảm bảo ứng dụng Single Page Application hoạt động đúng khi người dùng truy cập trực tiếp vào các đường dẫn con.

3 CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Phân tích yêu cầu

3.1.1 Yêu cầu chức năng

Các yêu cầu chức năng được xác định dựa trên phân tích nghiệp vụ quản lý công việc theo mô hình Kanban, bao gồm các nhóm chức năng chính sau:

Nhóm 1: Xác thực và quản lý tài khoản

Mã	Tên chức năng	Mô tả
FR-01	Đăng ký tài khoản	Người dùng nhập họ tên đầy đủ, địa chỉ email và mật khẩu để tạo tài khoản mới. Hệ thống gửi mã OTP xác nhận về email sau khi đăng ký thành công.
FR-02	Xác nhận email	Người dùng nhập mã OTP để kích hoạt tài khoản. Hỗ trợ chức năng gửi lại mã nếu mã đã hết hạn.
FR-03	Đăng nhập	Người dùng xác thực bằng email và mật khẩu. Sau khi xác thực thành công, access token và refresh token được lưu trong HttpOnly cookie.
FR-04	Đăng xuất	Phiên làm việc được kết thúc, cookie xác thực bị xóa khỏi trình duyệt.
FR-05	Làm mới phiên làm việc	Tự động làm mới access token bằng refresh token khi token hết hạn, đảm bảo phiên làm việc liên tục.
FR-32	Đổi mật khẩu	Người dùng thay đổi mật khẩu hiện tại thông qua xác thực với Amazon Cognito.
FR-33	Cập nhật hồ sơ cá nhân	Người dùng chỉnh sửa thông tin cá nhân như họ tên và ảnh đại diện.

Bảng 1: Yêu cầu chức năng nhóm 1

Nhóm 2: Quản lý Workspace

Mã	Tên chức năng	Mô tả
FR-06	Tạo Workspace	Người dùng tạo không gian làm việc mới với tên, loại và mô tả.
FR-07	Xem danh sách Workspace	Hệ thống hiển thị toàn bộ Workspace mà người dùng sở hữu hoặc là thành viên.
FR-08	Cập nhật Workspace	Người dùng chỉnh sửa tên, mô tả của Workspace.
FR-09	Quản lý thành viên Workspace	Thêm thành viên vào Workspace bằng địa chỉ email.
FR-30	Xóa Workspace	Người dùng (chủ sở hữu) xóa vĩnh viễn Workspace cùng toàn bộ Board, List và Card bên trong.

Bảng 2: Yêu cầu chức năng nhóm 2

Nhóm 3: Quản lý Board

Mã	Tên chức năng	Mô tả
FR-10	Tạo Board	Tạo bảng công việc mới trong Workspace với tiêu đề và hình nền tùy chọn.
FR-11	Xem Board	Tải và hiển thị toàn bộ danh sách List và Card trong Board theo mô hình Kanban.
FR-12	Cập nhật Board	Chỉnh sửa tiêu đề và hình nền của Board.
FR-13	Quản lý thành viên Board	Thêm thành viên từ Workspace vào Board với vai trò nhất định (ADMIN hoặc MEMBER).
FR-31	Xóa Board	Người dùng có quyền ADMIN xóa vĩnh viễn Board cùng toàn bộ List và Card bên trong.

Bảng 3: Yêu cầu chức năng nhóm 3

Nhóm 4: Quản lý List

Mã	Tên chức năng	Mô tả
FR-14	Tạo List	Tạo cột danh sách công việc mới trong Board.
FR-15	Cập nhật List	Đổi tên hoặc thay đổi vị trí của List trong Board.
FR-16	Xóa List	Xóa List và toàn bộ Card bên trong.

Bảng 4: Yêu cầu chức năng nhóm 4

Nhóm 5: Quản lý Card

Mã	Tên chức năng	Mô tả
FR-17	Tạo Card	Tạo thẻ công việc mới trong một List cụ thể với tiêu đề ban đầu.
FR-18	Xem chi tiết Card	Mở modal hiển thị đầy đủ thông tin Card bao gồm tiêu đề, mô tả, ngày bắt đầu, ngày đến hạn, thành viên và danh sách kiểm tra.
FR-19	Cập nhật Card	Chỉnh sửa tiêu đề, mô tả, ngày bắt đầu, ngày đến hạn và trạng thái hoàn thành của Card.
FR-20	Xóa Card	Xóa thẻ công việc khỏi hệ thống.
FR-22	Gán thành viên vào Card	Gán hoặc hủy gán thành viên chịu trách nhiệm thực hiện Card. Avatar thành viên được hiển thị trực tiếp trên Card.

Bảng 5: Yêu cầu chức năng nhóm 5

Nhóm 6: Kéo thả (Drag and Drop)

Mã	Tên chức năng	Mô tả
FR-23	Kéo thả Card	Di chuyển Card sang List khác hoặc sắp xếp lại thứ tự Card trong cùng một List; cập nhật <code>position</code> (và <code>listId</code> nếu đổi List) đồng bộ với Backend.
FR-24	Kéo thả List trong Board	Thay đổi thứ tự hiển thị của các List trong Board.

Bảng 6: Yêu cầu chức năng nhóm 6

Nhóm 7: Quản lý Checklist

Mã	Tên chức năng	Mô tả
FR-26	Tạo Checklist	Tạo danh sách kiểm tra mới trong Card.
FR-27	Cập nhật Checklist	Đổi tên Checklist.
FR-28	Xóa Checklist	Xóa Checklist và toàn bộ ChecklistItem liên quan.
FR-29	Quản lý ChecklistItem	Thêm, cập nhật trạng thái hoàn thành và xóa các hạng mục công việc con. Hiển thị thanh tiến độ (progress bar) theo tỷ lệ phần trăm hoàn thành.

Bảng 7: Yêu cầu chức năng nhóm 7

3.1.2 Yêu cầu phi chức năng

- **Bảo mật:** Toàn bộ giao tiếp giữa Client và Backend được thực hiện qua HTTP/HTTPS. Access token và refresh token được lưu trong HttpOnly cookie, ngăn chặn truy cập từ JavaScript phía Client (chống tấn công XSS). Xác thực người dùng được ủy thác cho Amazon Cognito. Các tài nguyên Backend và cơ sở dữ liệu được cách ly trong mạng nội bộ VPC.
- **Tính sẵn sàng:** Hệ thống được triển khai trên hạ tầng AWS với cân bằng tải qua ALB, đảm bảo tính sẵn sàng cao và khả năng phục hồi khi có sự cố.
- **Khả năng mở rộng:** Kiến trúc tách biệt Frontend và Backend cho phép mở rộng độc lập từng thành phần. Sử dụng Amazon RDS và EC2 giúp dễ dàng nâng cấp tài nguyên khi lưu lượng tăng.
- **Tính nhất quán dữ liệu:** Mọi thao tác kéo thả cập nhật trường position trong cơ sở dữ liệu, đảm bảo thứ tự hiển thị nhất quán giữa Frontend và Backend sau mỗi phiên làm việc.
- **Khả năng sử dụng:** Giao diện được thiết kế theo mô hình Kanban quen thuộc, hỗ trợ đầy đủ trên trình duyệt web hiện đại.

3.2 Phân rã chức năng

3.2.1 Xác thực và quản lý tài khoản

Thuộc tính	Nội dung
Use Case ID	FR-01
Tên Use Case	Đăng ký tài khoản
Actor	Người dùng chưa có tài khoản
Mục tiêu	Cho phép người dùng mới tạo tài khoản trong hệ thống.
Tiền điều kiện	Người dùng chưa có tài khoản trong hệ thống.
Hậu điều kiện	Tài khoản được khởi tạo trong Amazon Cognito; email OTP xác nhận được gửi.
Luồng chính	1. Người dùng nhập họ tên, email và mật khẩu. 2. Người dùng nhấn Đăng ký. 3. Hệ thống gọi Cognito tạo tài khoản. 4. Cognito gửi mã OTP về email.
Luồng thay thế	Email đã tồn tại trong hệ thống: Hệ thống trả về lỗi HTTP 409. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-02
Tên Use Case	Xác nhận email
Actor	Người dùng đã đăng ký nhưng chưa xác nhận email
Mục tiêu	Kích hoạt tài khoản bằng cách xác minh địa chỉ email qua mã OTP.
Tiền điều kiện	Tài khoản đã được tạo; mã OTP đã được gửi về email.
Hậu điều kiện	Tài khoản được kích hoạt; người dùng được chuyển hướng đến trang đăng nhập.
Luồng chính	1. Người dùng nhập mã OTP nhận được qua email. 2. Người dùng nhấn Xác nhận. 3. Hệ thống gửi yêu cầu xác nhận đến Cognito. 4. Cognito kích hoạt tài khoản. 5. Người dùng được chuyển hướng về trang đăng nhập.
Luồng thay thế	Mã OTP sai hoặc hết hạn: Hệ thống trả về lỗi; người dùng có thể yêu cầu gửi lại mã. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-03
Tên Use Case	Đăng nhập
Actor	Người dùng đã có tài khoản
Mục tiêu	Xác thực danh tính người dùng và cấp phiên làm việc.
Tiền điều kiện	Người dùng đã có tài khoản và đã xác nhận email.
Hậu điều kiện	Access token và refresh token được lưu trong HttpOnly cookie; người dùng được chuyển hướng đến Dashboard.
Luồng chính	1. Người dùng nhập email và mật khẩu. 2. Người dùng nhấn Đăng nhập. 3. Backend gọi Cognito xác thực. 4. Cognito trả về JWT. 5. Backend đặt token vào HttpOnly cookie. 6. Người dùng được chuyển hướng đến Dashboard.
Luồng thay thế	Email hoặc mật khẩu không đúng: Hệ thống trả về thông báo lỗi xác thực. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-04
Tên Use Case	Đăng xuất
Actor	Người dùng đã đăng nhập
Mục tiêu	Kết thúc phiên làm việc của người dùng.
Tiền điều kiện	Người dùng đang có phiên làm việc hợp lệ.
Hậu điều kiện	Cookie xác thực bị xóa khỏi trình duyệt; người dùng được chuyển hướng về trang đăng nhập.
Luồng chính	1. Người dùng nhấn nút Đăng xuất. 2. Backend xóa HttpOnly cookie. 3. Người dùng được chuyển hướng về trang đăng nhập.
Luồng thay thế	Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-05
Tên Use Case	Làm mới phiên làm việc
Actor	Hệ thống (tự động)
Mục tiêu	Duy trì phiên làm việc liên tục khi access token hết hạn.
Tiền điều kiện	Người dùng đang đăng nhập; access token đã hết hạn nhưng refresh token còn hợp lệ.
Hậu điều kiện	Access token mới được cấp và lưu vào cookie; phiên làm việc được duy trì.
Luồng chính	1. Frontend phát hiện access token hết hạn. 2. Frontend tự động gửi yêu cầu làm mới với refresh token. 3. Backend gọi Cognito để làm mới token. 4. Cookie được cập nhật với access token mới.
Luồng thay thế	Refresh token hết hạn: Người dùng bị đăng xuất và chuyển hướng về trang đăng nhập. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-32
Tên Use Case	Đổi mật khẩu
Actor	Người dùng đã đăng nhập
Mục tiêu	Cho phép người dùng thay đổi mật khẩu tài khoản hiện tại.
Tiền điều kiện	Người dùng đã đăng nhập và có phiên làm việc hợp lệ.
Hậu điều kiện	Mật khẩu mới được cập nhật trong Amazon Cognito.
Luồng chính	1. Người dùng truy cập popup đổi mật khẩu. 2. Người dùng nhập mật khẩu hiện tại và mật khẩu mới. 3. Hệ thống xác thực mật khẩu hiện tại với Cognito. 4. Cognito cập nhật mật khẩu mới. 5. Hệ thống hiển thị thông báo đổi mật khẩu thành công.
Luồng thay thế	Mật khẩu hiện tại không đúng: Hệ thống trả về thông báo lỗi xác thực. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-33
Tên Use Case	Cập nhật hồ sơ cá nhân
Actor	Người dùng đã đăng nhập
Mục tiêu	Cho phép người dùng chỉnh sửa thông tin hồ sơ cá nhân.
Tiền điều kiện	Người dùng đã đăng nhập và có phiên làm việc hợp lệ.
Hậu điều kiện	Thông tin hồ sơ được cập nhật trong cơ sở dữ liệu; giao diện phản ánh thay đổi.
Luồng chính	1. Người dùng truy cập trang hồ sơ cá nhân. 2. Người dùng chỉnh sửa thông tin (phone, địa chỉ, giới thiệu bản thân). 3. Người dùng nhấn Lưu. 4. Backend xác thực dữ liệu và cập nhật bản ghi. 5. Hệ thống hiển thị thông báo cập nhật thành công.
Luồng thay thế	Dữ liệu không hợp lệ (tên trống): Hệ thống trả về thông báo lỗi. Lỗi không xác định: Trả về lỗi HTTP 500.

3.2.2 Quản lý Workspace

Thuộc tính	Nội dung
Use Case ID	FR-06
Tên Use Case	Tạo Workspace
Actor	Người dùng đã đăng nhập
Mục tiêu	Khởi tạo không gian làm việc mới để tổ chức các dự án liên quan.
Tiền điều kiện	Người dùng đã đăng nhập.
Hậu điều kiện	Workspace mới được tạo; người dùng tự động trở thành ADMIN của Workspace.
Luồng chính	1. Người dùng nhập tên, loại và mô tả Workspace. 2. Người dùng nhấn Tạo. 3. Backend xác thực token và kiểm tra tên không trống. 4. Backend tạo bản ghi Workspace trong cơ sở dữ liệu. 5. Backend tự động thêm người dùng vào Workspace với vai trò ADMIN. 6. Workspace mới hiển thị trong Dashboard.
Luồng thay thế	Tên Workspace trống: Hệ thống trả về lỗi HTTP 400. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-07
Tên Use Case	Xem danh sách Workspace
Actor	Người dùng đã đăng nhập
Mục tiêu	Hiển thị toàn bộ Workspace mà người dùng sở hữu hoặc tham gia.
Tiền điều kiện	Người dùng đã đăng nhập.
Hậu điều kiện	Danh sách Workspace được hiển thị trong Dashboard.
Luồng chính	1. Người dùng truy cập Dashboard. 2. Frontend gửi yêu cầu lấy danh sách Workspace. 3. Backend truy vấn các Workspace liên kết với người dùng. 4. Danh sách Workspace được hiển thị.
Luồng thay thế	Không có Workspace: Hệ thống hiển thị trạng thái trống. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-08
Tên Use Case	Cập nhật Workspace
Actor	Người dùng có vai trò ADMIN trong Workspace
Mục tiêu	Chỉnh sửa thông tin của Workspace hiện có.
Tiền điều kiện	Người dùng có vai trò ADMIN trong Workspace.
Hậu điều kiện	Thông tin Workspace được cập nhật trong cơ sở dữ liệu.
Luồng chính	1. Người dùng mở cài đặt Workspace. 2. Người dùng chỉnh sửa tên hoặc mô tả. 3. Người dùng nhấn Lưu. 4. Backend kiểm tra quyền ADMIN. 5. Bản ghi được cập nhật. 6. Giao diện phản ánh thay đổi.
Luồng thay thế	Người dùng không có quyền ADMIN: Hệ thống trả về HTTP 403. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-09
Tên Use Case	Quản lý thành viên Workspace
Actor	Người dùng có vai trò ADMIN trong Workspace
Mục tiêu	Thêm thành viên vào Workspace để cộng tác.
Tiền điều kiện	Người dùng có vai trò ADMIN; người được mời đã có tài khoản trong hệ thống.
Hậu điều kiện	Thành viên được thêm vào Workspace với vai trò MEMBER.
Luồng chính	1. Người dùng tìm kiếm thành viên qua email hoặc tên. 2. Người dùng chọn thành viên cần thêm. 3. Backend kiểm tra người dùng tồn tại và chưa là thành viên. 4. Bản ghi thành viên được tạo. 5. Danh sách thành viên được cập nhật.
Luồng thay thế	Người dùng đã là thành viên: Hệ thống trả về thông báo lỗi. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-30
Tên Use Case	Xóa Workspace
Actor	Chủ sở hữu Workspace
Mục tiêu	Xóa vĩnh viễn Workspace và toàn bộ dữ liệu liên quan.
Tiền điều kiện	Người dùng là chủ sở hữu của Workspace.
Hậu điều kiện	Workspace và toàn bộ Board, List, Card con bị xóa khỏi hệ thống.
Luồng chính	1. Người dùng mở cài đặt Workspace. 2. Người dùng chọn xóa Workspace. 3. Người dùng xác nhận thao tác. 4. Backend kiểm tra quyền (chỉ chủ sở hữu mới được phép). 5. Xóa cascade toàn bộ Board, List, Card và bản ghi thành viên. 6. Workspace không còn xuất hiện trong Dashboard.
Luồng thay thế	Người dùng không phải chủ sở hữu: Hệ thống trả về HTTP 403. Lỗi không xác định: Trả về lỗi HTTP 500.

3.2.3 Quản lý Board

Thuộc tính	Nội dung
Use Case ID	FR-10
Tên Use Case	Tạo Board
Actor	Người dùng có vai trò ADMIN/MEMBER trong Workspace
Mục tiêu	Tạo bảng công việc mới đại diện cho một dự án cụ thể trong Workspace.
Tiền điều kiện	Người dùng có quyền ADMIN/MEMBER trong Workspace.
Hậu điều kiện	Board mới được tạo và hiển thị trong Workspace.
Luồng chính	1. Người dùng chọn tạo Board mới trong Workspace. 2. Người dùng nhập tên và chọn hình nền. 3. Backend kiểm tra Workspace tồn tại và quyền truy cập. 4. Board được tạo trong cơ sở dữ liệu. 5. Board mới hiển thị trong Workspace.
Luồng thay thế	Workspace không tồn tại: HTTP 404. Người dùng không có quyền: HTTP 403. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-11
Tên Use Case	Xem Board
Actor	Người dùng là thành viên của Board
Mục tiêu	Tải và hiển thị toàn bộ nội dung Board theo mô hình Kanban.
Tiền điều kiện	Người dùng là thành viên của Board.
Hậu điều kiện	Toàn bộ List và Card được hiển thị theo thứ tự position.
Luồng chính	1. Người dùng nhấn vào Board. 2. Frontend gửi yêu cầu lấy dữ liệu Board. 3. Backend xác thực quyền truy cập. 4. Backend truy vấn Board, List và Card. 5. Dữ liệu được hiển thị dạng Kanban.
Luồng thay thế	Board không tồn tại: HTTP 404. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-12
Tên Use Case	Cập nhật Board
Actor	Người dùng có vai trò ADMIN hoặc MEMBER trong Board
Mục tiêu	Chỉnh sửa tên hoặc hình nền của Board.
Tiền điều kiện	Người dùng có quyền ADMIN hoặc MEMBER trong Board.
Hậu điều kiện	Thông tin Board được cập nhật trong cơ sở dữ liệu.
Luồng chính	1. Người dùng mở cài đặt Board. 2. Người dùng chỉnh sửa tên hoặc hình nền. 3. Người dùng nhấn Lưu. 4. Backend kiểm tra quyền. 5. Bản ghi được cập nhật. 6. Giao diện phản ánh thay đổi.
Luồng thay thế	Người dùng không có quyền: Hệ thống trả về HTTP 403. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-13
Tên Use Case	Quản lý thành viên Board
Actor	Người dùng có vai trò ADMIN trong Board
Mục tiêu	Thêm thành viên của ứng dụng vào Board với vai trò nhất định.
Tiền điều kiện	Người dùng có quyền ADMIN trong Board; người được mời là thành viên của Workspace.
Hậu điều kiện	Thành viên được thêm vào Board với vai trò được chỉ định.
Luồng chính	1. Người dùng mở quản lý thành viên Board. 2. Người dùng tìm kiếm thành viên. 3. Người dùng chọn thành viên và vai trò. 4. Backend kiểm tra quyền và tư cách thành viên Workspace. 5. Bản ghi trong bảng board_member được tạo. 6. Danh sách thành viên được cập nhật.
Luồng thay thế	Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-31
Tên Use Case	Xóa Board
Actor	Người dùng có vai trò ADMIN trong Board
Mục tiêu	Xóa vĩnh viễn Board và toàn bộ dữ liệu liên quan.
Tiền điều kiện	Người dùng có vai trò ADMIN trong Board.
Hậu điều kiện	Board và toàn bộ List, Card, Checklist con bị xóa khỏi hệ thống.
Luồng chính	1. Người dùng mở cài đặt Board. 2. Người dùng chọn xóa Board. 3. Người dùng xác nhận thao tác. 4. Backend kiểm tra quyền ADMIN. 5. Xóa cascade toàn bộ List, Card, Checklist, ChecklistItem và bản ghi thành viên. 6. Board không còn xuất hiện trong Workspace.
Luồng thay thế	Người dùng không có quyền ADMIN: Hệ thống trả về HTTP 403. Lỗi không xác định: Trả về lỗi HTTP 500.

3.2.4 Quản lý List

Thuộc tính	Nội dung
Use Case ID	FR-14
Tên Use Case	Tạo List
Actor	Người dùng có vai trò ADMIN hoặc MEMBER trong Board
Mục tiêu	Tạo cột danh sách công việc mới trong Board.
Tiền điều kiện	Người dùng có quyền ADMIN hoặc MEMBER trong Board.
Hậu điều kiện	List mới được tạo và hiển thị trong Board.
Luồng chính	1. Người dùng nhấn thêm List mới. 2. Người dùng nhập tên List. 3. Backend xác thực quyền truy cập Board. 4. Bản ghi List được tạo với position được cung cấp. 5. List mới hiển thị trong Board.
Luồng thay thế	Người dùng không có quyền: Hệ thống trả về HTTP 403. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-15
Tên Use Case	Cập nhật List
Actor	Người dùng có vai trò ADMIN hoặc MEMBER trong Board
Mục tiêu	Đổi tên hoặc thay đổi vị trí của List trong Board.
Tiền điều kiện	List tồn tại; người dùng có quyền trong Board.
Hậu điều kiện	Thông tin List được cập nhật; giao diện Board phản ánh thay đổi.
Luồng chính	1. Người dùng chỉnh sửa tên List hoặc kéo thả để thay đổi vị trí. 2. Backend xác thực quyền. 3. Bản ghi được cập nhật. 4. Giao diện phản ánh thay đổi.
Luồng thay thế	List không tồn tại: Hệ thống trả về HTTP 404. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-16
Tên Use Case	Xóa List
Actor	Người dùng có vai trò ADMIN hoặc MEMBER trong Board
Mục tiêu	Xóa List và toàn bộ Card bên trong khối Board.
Tiền điều kiện	List tồn tại; người dùng có quyền ADMIN hoặc MEMBER trong Board.
Hậu điều kiện	List và tất cả Card con bị xóa khỏi cơ sở dữ liệu.
Luồng chính	1. Người dùng chọn xóa List. 2. Người dùng xác nhận thao tác. 3. Backend kiểm tra quyền. 4. Xóa cascade List và Card. 5. Giao diện Board được cập nhật.
Luồng thay thế	Người dùng không có quyền: Hệ thống trả về HTTP 403. Lỗi không xác định: Trả về lỗi HTTP 500.

3.2.5 Quản lý Card

Thuộc tính	Nội dung
Use Case ID	FR-17
Tên Use Case	Tạo Card
Actor	Người dùng có vai trò ADMIN hoặc MEMBER trong Board
Mục tiêu	Tạo thẻ công việc mới trong một List cụ thể.
Tiền điều kiện	List tồn tại; người dùng có quyền trong Board.
Hậu điều kiện	Card mới được tạo và hiển thị trong List.
Luồng chính	1. Người dùng nhấn thêm Card trong List. 2. Người dùng nhập tiêu đề Card. 3. Backend xác thực quyền truy cập Board. 4. Bản ghi Card được tạo với position được cung cấp. 5. Card mới hiển thị trong List.
Luồng thay thế	List không tồn tại: HTTP 404. Người dùng không có quyền: HTTP 403. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-18
Tên Use Case	Xem chi tiết Card
Actor	Người dùng là thành viên của Board
Mục tiêu	Hiển thị đầy đủ thông tin của Card trong modal.
Tiền điều kiện	Card tồn tại; người dùng có quyền truy cập Board.
Hậu điều kiện	Thông tin Card được hiển thị đầy đủ trong modal.
Luồng chính	1. Người dùng nhấn vào Card. 2. Frontend gửi yêu cầu lấy chi tiết Card. 3. Backend truy vấn Card cùng các quan hệ liên quan. 4. Modal hiển thị đầy đủ tiêu đề, mô tả, ngày, thành viên và Checklist.
Luồng thay thế	Card không tồn tại: Hệ thống trả về HTTP 404. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-19
Tên Use Case	Cập nhật Card
Actor	Người dùng có vai trò ADMIN hoặc MEMBER trong Board
Mục tiêu	Chỉnh sửa thông tin chi tiết của Card.
Tiền điều kiện	Card tồn tại; người dùng có quyền trong Board.
Hậu điều kiện	Card được cập nhật trong cơ sở dữ liệu; giao diện phản ánh thay đổi.
Luồng chính	1. Người dùng mở chi tiết Card. 2. Người dùng chỉnh sửa tiêu đề, mô tả, ngày bắt đầu, ngày đến hạn hoặc trạng thái. 3. Backend kiểm tra quyền. 4. Bản ghi được cập nhật. 5. Giao diện phản ánh thay đổi.
Luồng thay thế	Người dùng không có quyền: Hệ thống trả về HTTP 403. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-20
Tên Use Case	Xóa Card
Actor	Người dùng có vai trò ADMIN hoặc MEMBER trong Board
Mục tiêu	Xóa thẻ công việc khỏi hệ thống.
Tiền điều kiện	Card tồn tại; người dùng có quyền trong Board.
Hậu điều kiện	Card bị xóa khỏi cơ sở dữ liệu và giao diện Board.
Luồng chính	1. Người dùng chọn xóa Card. 2. Người dùng xác nhận thao tác. 3. Backend kiểm tra quyền. 4. Bản ghi Card bị xóa. 5. Giao diện Board được cập nhật.
Luồng thay thế	Người dùng không có quyền: Hệ thống trả về HTTP 403. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-22
Tên Use Case	Gán thành viên vào Card
Actor	Người dùng có vai trò ADMIN hoặc MEMBER trong Board
Mục tiêu	Chỉ định thành viên chịu trách nhiệm thực hiện Card.
Tiền điều kiện	Thành viên là thành viên của Board.
Hậu điều kiện	Bản ghi trong bảng card_member được tạo hoặc xóa; avatar thành viên hiển thị trên Card.
Luồng chính	1. Người dùng mở chi tiết Card. 2. Người dùng chọn thành viên cần gán. 3. Backend kiểm tra thành viên thuộc Board. 4. Bản ghi card_member được tạo. 5. Avatar thành viên hiển thị trên Card.
Luồng thay thế	Hủy gán: Người dùng nhấn xóa thành viên khỏi Card; bản ghi card_member bị xóa. Lỗi không xác định: Trả về lỗi HTTP 500.

3.2.6 Kéo thả (Drag and Drop)

Thuộc tính	Nội dung
Use Case ID	FR-23
Tên Use Case	Kéo thả Card
Actor	Người dùng có vai trò ADMIN hoặc MEMBER trong Board
Mục tiêu	Di chuyển Card sang List khác hoặc sắp xếp lại thứ tự Card trong cùng một List.
Tiền điều kiện	Người dùng có quyền trong Board; Board đang được hiển thị.
Hậu điều kiện	Card được cập nhật position (và listId nếu di chuyển sang List khác); giao diện phản ánh thay đổi tức thì.
Luồng chính	1. Người dùng kéo Card đến vị trí đích (trong cùng List hoặc sang List khác). 2. Frontend cập nhật giao diện ngay lập tức (optimistic update). 3. Frontend gửi PATCH /api/v1/cards/:id với position mới (kèm listId nếu di chuyển sang List khác). 4. Backend cập nhật cơ sở dữ liệu.
Luồng thay thế	Yêu cầu API thất bại: Giao diện hoàn nguyên về trạng thái ban đầu. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-24
Tên Use Case	Kéo thả List trong Board
Actor	Người dùng có vai trò ADMIN hoặc MEMBER trong Board
Mục tiêu	Thay đổi thứ tự hiển thị của các List trong Board.
Tiền điều kiện	Người dùng có quyền trong Board; Board đang được hiển thị.
Hậu điều kiện	Thứ tự List được cập nhật; giao diện Board phản ánh thay đổi tức thì.
Luồng chính	1. Người dùng kéo List và thả vào vị trí mới. 2. Frontend tính toán position mới. 3. Frontend cập nhật giao diện ngay (optimistic update). 4. Frontend gửi PATCH /api/v1/lists/:id với position mới. 5. Backend cập nhật cơ sở dữ liệu.
Luồng thay thế	Yêu cầu API thất bại: Giao diện hoàn nguyên về trạng thái ban đầu. Lỗi không xác định: Trả về lỗi HTTP 500.

3.2.7 Quản lý Checklist

Thuộc tính	Nội dung
Use Case ID	FR-26
Tên Use Case	Tạo Checklist
Actor	Người dùng có vai trò ADMIN hoặc MEMBER trong Board
Mục tiêu	Tạo danh sách kiểm tra mới trong Card để theo dõi công việc con.
Tiền điều kiện	Card tồn tại; người dùng có quyền trong Board.
Hậu điều kiện	Checklist mới được tạo và hiển thị trong modal Card.
Luồng chính	1. Người dùng mở chi tiết Card. 2. Người dùng nhấn thêm Checklist. 3. Người dùng nhập tên Checklist. 4. Backend xác thực quyền. 5. Bản ghi Checklist được tạo. 6. Checklist hiển thị trong modal Card.
Luồng thay thế	Người dùng không có quyền: Hệ thống trả về HTTP 403. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-27
Tên Use Case	Cập nhật Checklist
Actor	Người dùng có vai trò ADMIN hoặc MEMBER trong Board
Mục tiêu	Đổi tên Checklist.
Tiền điều kiện	Checklist tồn tại; người dùng có quyền trong Board.
Hậu điều kiện	Tên Checklist được cập nhật.
Luồng chính	1. Người dùng nhấn chỉnh sửa tên Checklist. 2. Người dùng nhập tên mới. 3. Backend cập nhật bản ghi. 4. Giao diện phản ánh thay đổi.
Luồng thay thế	Tên trống: Không cho phép lưu. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-28
Tên Use Case	Xóa Checklist
Actor	Người dùng có vai trò ADMIN hoặc MEMBER trong Board
Mục tiêu	Xóa Checklist và toàn bộ ChecklistItem liên quan.
Tiền điều kiện	Checklist tồn tại; người dùng có quyền trong Board.
Hậu điều kiện	Checklist và các ChecklistItem bị xóa khỏi cơ sở dữ liệu.
Luồng chính	1. Người dùng nhấn xóa Checklist. 2. Người dùng xác nhận thao tác. 3. Backend xóa cascade Checklist và ChecklistItem. 4. Giao diện modal Card được cập nhật.
Luồng thay thế	Người dùng không có quyền: Hệ thống trả về HTTP 403. Lỗi không xác định: Trả về lỗi HTTP 500.

Thuộc tính	Nội dung
Use Case ID	FR-29
Tên Use Case	Quản lý ChecklistItem
Actor	Người dùng có vai trò ADMIN hoặc MEMBER trong Board
Mục tiêu	Thêm, cập nhật và xóa các hạng mục công việc con; theo dõi tiến độ hoàn thành.
Tiền điều kiện	Checklist tồn tại; người dùng có quyền trong Board.
Hậu điều kiện	ChecklistItem được quản lý; thanh tiến độ phản ánh tỷ lệ phần trăm hoàn thành.
Luồng chính	1. Thêm: Người dùng nhấn thêm item, nhập tên và nhấn Lưu; hệ thống gọi POST /api/v1/checklist-items. 2. Cập nhật: Người dùng tích hoàn thành hoặc chỉnh sửa tên; hệ thống gọi PATCH /api/v1/checklist-items/:id. 3. Xóa: Người dùng nhấn xóa item; hệ thống gọi DELETE /api/v1/checklist-items/:id. 4. Frontend tính lại tỷ lệ hoàn thành và cập nhật thanh tiến độ.
Luồng thay thế	Tên item trống: Không cho phép thêm. Lỗi không xác định: Trả về lỗi HTTP 500.

3.3 Phân tích dữ liệu

3.3.1 Các thực thể chính

Hệ thống xoay quanh sáu thực thể nghiệp vụ chính được tổ chức theo cấu trúc phân cấp:

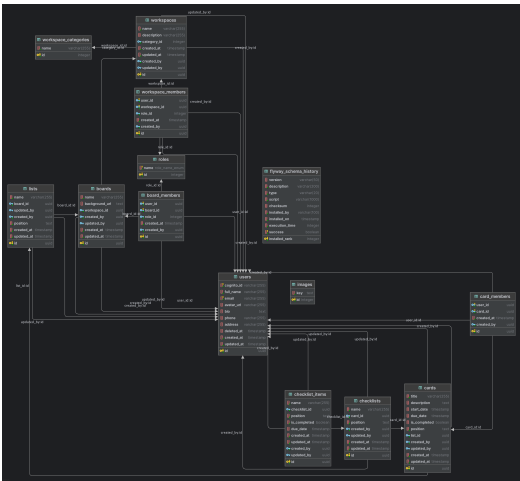
- **User (Người dùng):** Đại diện cho tài khoản người dùng trong hệ thống. Thông tin danh tính và xác thực được quản lý bởi Amazon Cognito; hệ thống nội bộ lưu trữ thông tin bổ sung như họ tên, ảnh đại diện và mã định danh Cognito.
- **Workspace (Không gian làm việc):** Đơn vị tổ chức cấp cao nhất, quản lý tập trung nhiều Board liên quan. Mỗi Workspace có một người sở hữu (owner) và nhiều thành viên.
- **Board (Bảng công việc):** Đại diện cho một dự án cụ thể trong Workspace. Mỗi Board chứa nhiều List và có danh sách thành viên riêng.
- **List (Danh sách):** Cột trạng thái trong Board, thể hiện giai đoạn thực hiện của công việc. Thứ tự các List được quản lý qua trường `position`.
- **Card (Thẻ công việc):** Đơn vị công việc cơ bản, chứa thông tin chi tiết về nhiệm vụ cần thực hiện. Card thuộc về một List và có thứ tự xác định bằng `position`.
- **Checklist:** Danh sách kiểm tra trong Card, hỗ trợ theo dõi các công việc con. Mỗi Checklist có nhiều ChecklistItem.
- **ChecklistItem:** Hạng mục cụ thể trong Checklist, có thể được đánh dấu hoàn thành hoặc chưa hoàn thành, và có thể thiết lập thời hạn riêng.

3.3.2 Quan hệ giữa các thực thể

- Một **User** có thể sở hữu nhiều **Workspace** và là thành viên của nhiều **Workspace** và **Board** khác.
- Một **Workspace** chứa nhiều **Board** và có nhiều **User** làm thành viên (quan hệ nhiều-nhiều qua bảng `workspace_member`).
- Một **Board** chứa nhiều **List** và có nhiều **User** làm thành viên với các vai trò khác nhau (quan hệ nhiều-nhiều qua bảng `board_member`).
- Một **List** chứa nhiều **Card**; các Card được sắp xếp theo thứ tự `position`.
- Một **Card** có thể chứa nhiều **Checklist** và được gán cho nhiều **User** (quan hệ nhiều-nhiều qua bảng `card_member`).
- Một **Checklist** chứa nhiều **ChecklistItem**.

3.4 Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu PostgreSQL được thiết kế bao gồm các kiểu liệt kê (ENUM), bảng tra cứu và bảng nghiệp vụ. Mỗi bảng được mô tả chi tiết qua cấu trúc cột, kiểu dữ liệu và ràng buộc tương ứng.



Hình 6: Thiết kế cơ sở dữ liệu

3.4.1 Kiểu liệt kê

Hệ thống định nghĩa kiểu liệt kê `role_name_enum` để chuẩn hóa các vai trò người dùng trong toàn bộ hệ thống.

Giá trị	Mô tả
ADMIN	Quản trị viên, có toàn quyền quản lý trong Workspace hoặc Board.
MEMBER	Thành viên thông thường.
VIEWER	Người xem, chỉ có quyền đọc dữ liệu.

Bảng 8: Kiểu liệt kê `role_name_enum`

3.4.2 Bảng tra cứu

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	Khóa chính, NOT NULL	Mã vai trò.
name	role_name_enum	NOT NULL, UNIQUE	Tên vai trò (ADMIN, MEMBER, VIEWER).

Bảng 9: Bảng roles

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	Khóa chính, NOT NULL	Mã hình ảnh.
key	TEXT	NOT NULL	Đường dẫn tương đối đến tệp hình nền Board.

Bảng 10: Bảng images

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	Khóa chính	Mã loại Workspace.
name	VARCHAR(255)	NOT NULL	Tên loại Workspace (ví dụ: Sales CRM, IT, Education...).

Bảng 11: Bảng workspace_categories

3.4.3 Bảng nghiệp vụ

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	Khóa chính, NOT NULL	Mã định danh nội bộ.
cognito_id	VARCHAR(255)	NOT NULL, UNIQUE	Mã định danh từ Amazon Cognito.
full_name	VARCHAR(255)	NOT NULL	Họ tên đầy đủ.
email	VARCHAR(255)	NOT NULL, UNIQUE	Địa chỉ email.
avatar_url	VARCHAR(255)	NULL	Đường dẫn ảnh đại diện lưu trên S3.
bio	TEXT	NULL	Giới thiệu bản thân.
phone	VARCHAR(255)	NULL	Số điện thoại.
address	VARCHAR(255)	NULL	Địa chỉ.
deleted_at	TIMESTAMP	NULL	Thời điểm xóa mềm tài khoản.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm tạo bản ghi.
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm cập nhật gần nhất.

Bảng 12: Bảng users

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	Khóa chính, NOT NULL	Mã định danh.
name	VARCHAR(255)	NOT NULL	Tên Workspace.
description	VARCHAR(255)	NULL	Mô tả Workspace.
category_id	INT	NOT NULL, FK → workspace_categories	Loại Workspace.
created_by	UUID	NOT NULL, FK → users	Người tạo Workspace.
updated_by	UUID	NULL, FK → users	Người cập nhật gần nhất.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm tạo bản ghi.
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm cập nhật gần nhất.

Bảng 13: Bảng **workspaces**

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	Khóa chính, NOT NULL	Mã định danh.
user_id	UUID	NOT NULL, FK → users	Thành viên được thêm vào Workspace.
workspace_id	UUID	NOT NULL, FK → workspaces	Workspace chứa thành viên.
role_id	INT	NOT NULL, FK → roles	Vai trò của thành viên trong Workspace.
created_by	UUID	NOT NULL, FK → users	Người thêm thành viên.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm thêm thành viên.
Ràng buộc UNIQUE: (user_id, workspace_id)			

Bảng 14: Bảng **workspace_members** – quan hệ nhiều-nhiều giữa **users** và **workspaces**

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	Khóa chính, NOT NULL	Mã định danh.
name	VARCHAR(255)	NOT NULL	Tên Board.
background_url	TEXT	NULL	Đường dẫn hình nền.
workspace_id	UUID	NOT NULL, FK → workspaces	Workspace chứa Board.
created_by	UUID	NOT NULL, FK → users	Người tạo Board.
updated_by	UUID	NULL, FK → users	Người cập nhật gần nhất.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm tạo bản ghi.
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm cập nhật gần nhất.

Bảng 15: Bảng **boards**

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	Khóa chính, NOT NULL	Mã định danh.
user_id	UUID	NOT NULL, FK → users	Thành viên được thêm vào Board.
board_id	UUID	NOT NULL, FK → boards	Board chứa thành viên.
role_id	INT	NOT NULL, FK → roles	Vai trò trong Board (ADMIN, MEMBER, VIEWER).
created_by	UUID	NOT NULL, FK → users	Người thêm thành viên.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm thêm thành viên.
Ràng buộc UNIQUE: (user_id, board_id)			

Bảng 16: Bảng **board_members** – quan hệ nhiều-nhiều giữa **users** và **boards**

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	Khóa chính, NOT NULL	Mã định danh.
name	VARCHAR(255)	NOT NULL	Tên List.
position	TEXT	NOT NULL	Giá trị vị trí dùng để sắp xếp và xử lý kéo thả.
board_id	UUID	NOT NULL, FK → boards	Board chứa List.
created_by	UUID	NOT NULL, FK → users	Người tạo List.
updated_by	UUID	NULL, FK → users	Người cập nhật gần nhất.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm tạo bản ghi.
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm cập nhật gần nhất.

Bảng 17: Bảng lists

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	Khóa chính, NOT NULL	Mã định danh.
title	VARCHAR(255)	NOT NULL	Tiêu đề Card.
description	TEXT	NULL	Mô tả chi tiết.
position	TEXT	NOT NULL	Giá trị vị trí dùng để sắp xếp và xử lý kéo thả.
list_id	UUID	NOT NULL, FK → lists	List chứa Card.
start_date	TIMESTAMP	NULL	Ngày bắt đầu.
due_date	TIMESTAMP	NULL	Ngày đến hạn.
is_completed	BOOLEAN	DEFAULT FALSE	Trạng thái hoàn thành của Card.
created_by	UUID	NOT NULL, FK → users	Người tạo Card.
updated_by	UUID	NULL, FK → users	Người cập nhật gần nhất.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm tạo bản ghi.
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm cập nhật gần nhất.

Bảng 18: Bảng **cards**

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	Khóa chính, NOT NULL	Mã định danh.
user_id	UUID	NOT NULL, FK → users	Thành viên được gán vào Card.
card_id	UUID	NOT NULL, FK → cards	Card được gán thành viên.
created_by	UUID	NOT NULL, FK → users	Người gán thành viên.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm gán thành viên.
Ràng buộc UNIQUE: (user_id, card_id)			

Bảng 19: Bảng **card_members** – quan hệ nhiều-nhiều giữa **users** và **cards**

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	Khóa chính, NOT NULL	Mã định danh.
name	VARCHAR(255)	NOT NULL	Tên Checklist.
position	TEXT	NOT NULL	Vị trí hiển thị trong Card.
card_id	UUID	NOT NULL, FK → cards	Card chứa Checklist.
created_by	UUID	NOT NULL, FK → users	Người tạo Checklist.
updated_by	UUID	NULL, FK → users	Người cập nhật gần nhất.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm tạo bản ghi.
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm cập nhật gần nhất.

Bảng 20: Bảng **checklists**

Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	UUID	Khóa chính, NOT NULL	Mã định danh.
name	VARCHAR(255)	NOT NULL	Tên hạng mục công việc con.
position	TEXT	NOT NULL	Vị trí hiển thị trong Checklist.
is_completed	BOOLEAN	NOT NULL, DEFAULT FALSE	Trạng thái hoàn thành của hạng mục.
due_date	TIMESTAMP	NULL	Thời hạn riêng của hạng mục.
checklist_id	UUID	NOT NULL, FK → checklists	Checklist chứa hạng mục.
created_by	UUID	NOT NULL, FK → users	Người tạo hạng mục.
updated_by	UUID	NULL, FK → users	Người cập nhật gần nhất.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm tạo bản ghi.
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm cập nhật gần nhất.
Ràng buộc CHECK: is_completed IN (true, false)			

Bảng 21: Bảng `checklist_items`

3.5 Thiết kế tổng thể hệ thống

Hệ thống được triển khai trên nền tảng điện toán đám mây AWS với sự bao bọc trong một Mạng riêng ảo (VPC) nhằm tối ưu hóa tính bảo mật. Kiến trúc tổng thể được thiết kế với 9 thành phần chính và sự tương tác chặt chẽ như sau:

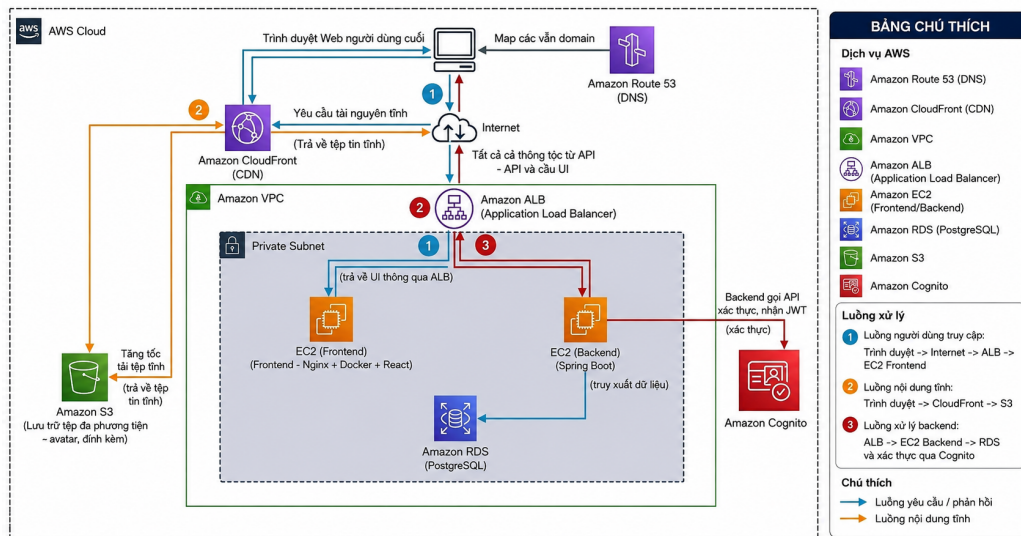
- **Client:** Đại diện cho trình duyệt web của người dùng cuối, là nơi giao diện ứng dụng được hiển thị và thực hiện các tương tác với hệ thống.
- **Amazon CloudFront:** Đóng vai trò là mạng phân phối nội dung (CDN). CloudFront *chỉ tương tác trực tiếp với Client và Amazon S3*, có nhiệm vụ tăng tốc độ tải các tệp tin tĩnh (hình ảnh, tài liệu) từ S3 đến người dùng một cách tối ưu thông qua bộ nhớ đệm (caching), giảm chi phí sử dụng S3.

- **Amazon S3:** Được sử dụng chuyên biệt làm kho lưu trữ các tệp đa phương tiện do người dùng tải lên (ví dụ: avatar, tệp đính kèm trong thẻ công việc). Tầng Backend đảm nhận hoàn toàn việc tải tệp tin lên S3 bằng cách tương tác trực tiếp với dịch vụ này; Client không có quyền đẩy trực tiếp tệp tin lên S3.
- **Amazon VPC (Virtual Private Cloud):** Cung cấp một môi trường mạng nội bộ bị cách ly với Internet. Toàn bộ các máy chủ EC2 (cả Frontend và Backend) cũng như cơ sở dữ liệu RDS đều được đặt bên trong VPC này (phân vùng private).
- **Amazon ALB (Application Load Balancer):** Là điểm truy cập (entry point) công khai duy nhất nhận lưu lượng truy cập từ Internet vào VPC. ALB tiếp nhận mọi request từ Client và phân phối tải vào các máy chủ EC2 (Frontend hoặc Backend) ở bên trong mạng private.
- **EC2 (Frontend – Nginx + Docker + React):** Ứng dụng ReactJS được đóng gói bằng Docker và triển khai thông qua web server Nginx trên máy chủ ảo EC2. Máy chủ này là một EC2 private, chỉ nhận lưu lượng HTTP/HTTPS từ ALB chuyển vào thay vì tiếp xúc trực tiếp với Internet.
- **EC2 (Backend – Spring Boot):** Máy chủ chạy ứng dụng API Spring Boot, xử lý các logic nghiệp vụ. Tương tự như Frontend, máy chủ này cũng là một EC2 private và chỉ nhận request nội bộ từ ALB.
- **Amazon Cognito:** Dịch vụ quản lý định danh. Trong kiến trúc này, Cognito chỉ tương tác trực tiếp với tầng Backend. Thay vì Client gọi trực tiếp Cognito, Backend sẽ chịu trách nhiệm gọi API của Cognito để thực hiện xác thực người dùng và nhận JWT.
- **Amazon RDS (PostgreSQL):** Hệ quản trị cơ sở dữ liệu quan hệ được triển khai trong phân vùng mạng nội bộ sâu nhất. Chỉ duy nhất tầng Backend (EC2 chạy Spring Boot) mới được phép tương tác trực tiếp với RDS để thực hiện truy xuất và lưu trữ dữ liệu; mọi nỗ lực truy cập từ các thành phần khác đều bị từ chối hoàn toàn.

Luồng hoạt động tổng thể của hệ thống diễn ra như sau:

1. **Tải giao diện ứng dụng:** Client gửi request → ALB → EC2 (Frontend – Nginx + React) → Trả về giao diện cho người dùng.
2. **Tải tài nguyên tĩnh (hình ảnh, tệp):** Client → CloudFront → S3 → Trả về Client.

3. **Xử lý nghiệp vụ và xác thực:** Client gọi API → ALB → EC2 (Backend – Spring Boot). Tại đây, nếu là tác vụ đăng nhập hoặc đăng ký, EC2 Backend sẽ giao tiếp với Cognito. Đối với các tác vụ liên quan đến dữ liệu hệ thống, EC2 Backend sẽ giao tiếp trực tiếp với RDS (PostgreSQL). Kết quả xử lý cuối cùng được Backend trả về qua ALB đến Client.



Hình 7: Kiến trúc tổng thể của hệ thống Nello

4 CHƯƠNG 4: TRIỂN KHAI VÀ KẾT QUẢ

4.1 Các chức năng đã hoàn thành

Sau 8 tuần thực hiện (từ 30/03 đến 24/05/2026), hệ thống Nello đã hoàn thành **27/31 chức năng** yêu cầu, đạt tỷ lệ **87,1%**.

STT	Mã FR	Tên chức năng
Nhóm 1: Xác thực và quản lý tài khoản		
1	FR-01	Đăng ký tài khoản
2	FR-02	Xác nhận email
3	FR-03	Đăng nhập
4	FR-04	Đăng xuất
5	FR-05	Làm mới phiên làm việc
6	FR-32	Đổi mật khẩu
7	FR-33	Cập nhật hồ sơ cá nhân
Nhóm 2: Quản lý Workspace		
8	FR-06	Tạo Workspace
9	FR-07	Xem danh sách Workspace
10	FR-09	Quản lý thành viên Workspace
Nhóm 3: Quản lý Board		
11	FR-10	Tạo Board
12	FR-11	Xem Board
13	FR-13	Quản lý thành viên Board
Nhóm 4: Quản lý List		
14	FR-14	Tạo List
15	FR-15	Cập nhật List
16	FR-16	Xóa List
Nhóm 5: Quản lý Card		
17	FR-17	Tạo Card
18	FR-18	Xem chi tiết Card
19	FR-19	Cập nhật Card
20	FR-20	Xóa Card
21	FR-22	Gán thành viên vào Card
Nhóm 6: Kéo thả (Drag and Drop)		
22	FR-23	Kéo thả Card
23	FR-24	Kéo thả List trong Board
Nhóm 7: Quản lý Checklist		

STT	Mã FR	Tên chức năng
24	FR-26	Tạo Checklist
25	FR-27	Cập nhật Checklist
26	FR-28	Xóa Checklist
27	FR-29	Quản lý ChecklistItem

Bảng 22: Danh sách chức năng đã hoàn thành (27/31 — 87,1%)

4.2 Kết quả đạt được

4.2.1 Thống kê tổng quát

Hạng mục	Kết quả
Thời gian thực hiện	8 tuần (30/03/2026 – 24/05/2026)
Số API endpoint hoàn thành	43 endpoint trên 14 nhóm module (toàn bộ Backend)
Số màn hình Frontend	6 màn hình chính
Số test case đã kiểm thử	Hơn 30 test case
Trạng thái Backend	Hoàn chỉnh
Trạng thái Frontend	Hoàn chỉnh các chức năng cốt lõi
Môi trường triển khai	Local development

Bảng 23: Tổng hợp kết quả thực hiện

4.2.2 Kết quả kiểm thử theo module

Quá trình kiểm thử được thực hiện theo từng tuần phát triển. Phần lớn các test case tập trung vào luồng thành công (happy case) và một số trường hợp lỗi cơ bản.

Module Authentication (W3) — 14 test case

Mã TC	Kịch bản	Kết quả
TC01–TC03	Đăng ký: hợp lệ, trùng email, thiếu trường bắt buộc	Pass
TC04–TC05	Xác nhận OTP: mã đúng, mã sai	Pass
TC06–TC09	Đăng nhập: hợp lệ, sai mật khẩu, chưa xác nhận email, kiểm tra cookie	Pass
TC10–TC12	Làm mới token: hợp lệ, thiếu token, token hết hạn	Pass
TC13–TC14	Gửi lại mã: user hợp lệ, user không tồn tại	Pass

Bảng 24: Kết quả kiểm thử module Authentication

Module Workspace & Board (W4) — 16 test case

Mã TC	Kịch bản	Kết quả
TC01–TC05	Workspace: tạo mới, thiếu tên, token không hợp lệ, lấy danh sách, danh sách rỗng	Pass
TC06–TC08	Board: tạo mới, Workspace không tồn tại, không có quyền	Pass
TC09–TC12	Board: cập nhật, không tồn tại, xóa, xóa không có quyền	Pass
TC13–TC16	List: tạo mới, Board không tồn tại, cập nhật, lấy danh sách	Pass

Bảng 25: Kết quả kiểm thử module Workspace & Board

Module Card & Checklist (W5) — 9 test case

Mã TC	Kịch bản	Kết quả
TC17–TC22	Card: tạo mới, lấy chi tiết, cập nhật, di chuyển (drag-drop), xóa, lấy danh sách theo Board	Pass
TC23–TC25	Checklist: tạo mới, tạo ChecklistItem, cập nhật trạng thái hoàn thành	Pass

Bảng 26: Kết quả kiểm thử module Card & Checklist

Kiểm thử tích hợp Frontend (W6) — 9 test case

Mã TC	Kịch bản	Kết quả
TC01–TC03	Luồng xác thực: đăng ký → xác nhận OTP → đăng nhập	Pass
TC04–TC06	Xác thực lỗi: user đã tồn tại, sai mật khẩu, mã OTP sai	Pass
TC07–TC09	Dashboard: tải giao diện, tạo Workspace, tạo Board	Pass

Bảng 27: Kết quả kiểm thử tích hợp Frontend (W6)

Kiểm thử giao diện Board (W7) — 8 test case

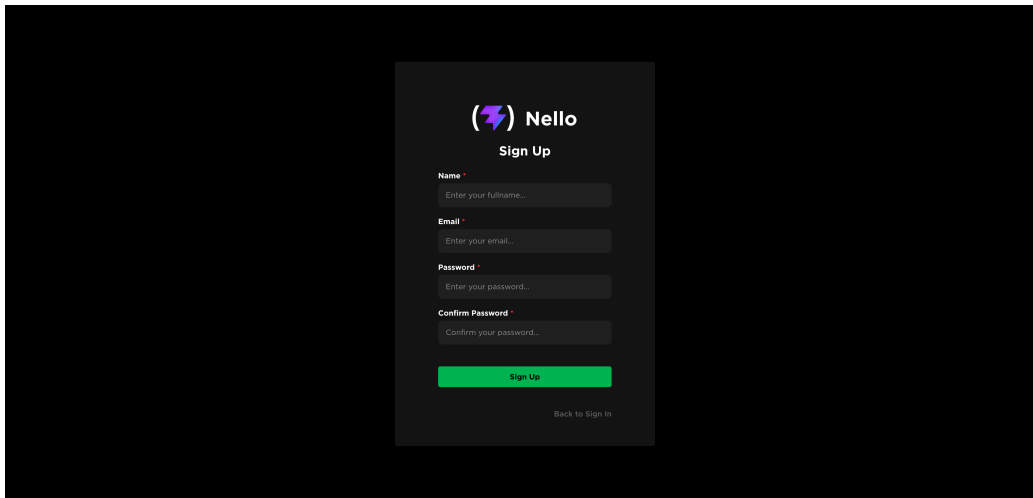
Mã TC	Kịch bản	Kết quả
TC01–TC02	Workspace Detail: tải giao diện, mời thành viên	Pass
TC04–TC08	Board Kanban: tải Board, tạo List, tạo Card, xóa Card, xóa List	Pass

Bảng 28: Kết quả kiểm thử giao diện Board (W7)

4.2.3 Các màn hình giao diện đã hoàn thành

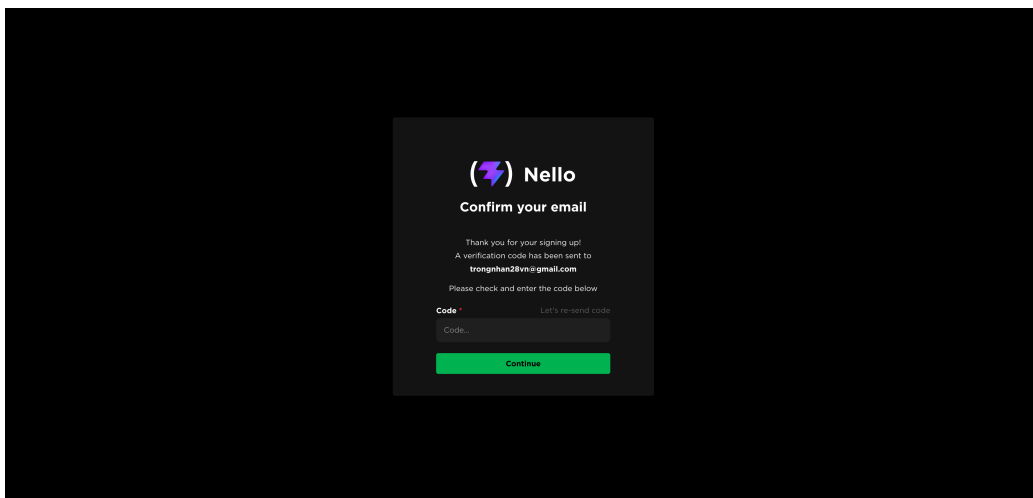
Hệ thống Frontend bao gồm 6 màn hình chính đã được phát triển và kiểm thử:

1. **Màn hình Đăng ký (/sign-up):** Form nhập họ tên, email, mật khẩu với validation phía Client; thông báo lỗi khi tài khoản đã tồn tại; tự động chuyển hướng sang màn hình xác nhận OTP sau khi đăng ký.



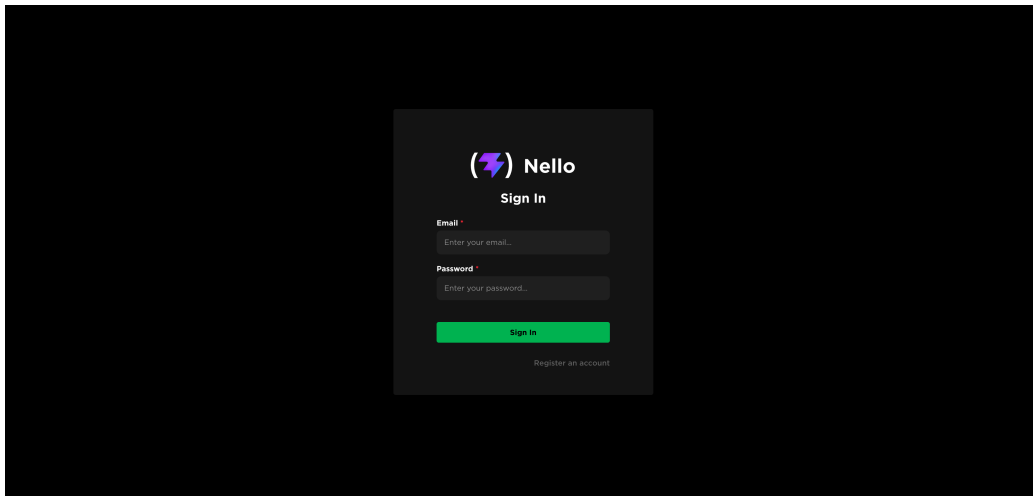
Hình 8: Màn hình Đăng ký tài khoản

2. **Màn hình Xác nhận OTP (/confirm-sign-up):** Ô nhập mã OTP 6 chữ số; nút gửi lại mã có bộ đếm thời gian; thông báo lỗi khi mã không hợp lệ; chuyển hướng về đăng nhập sau khi xác nhận.



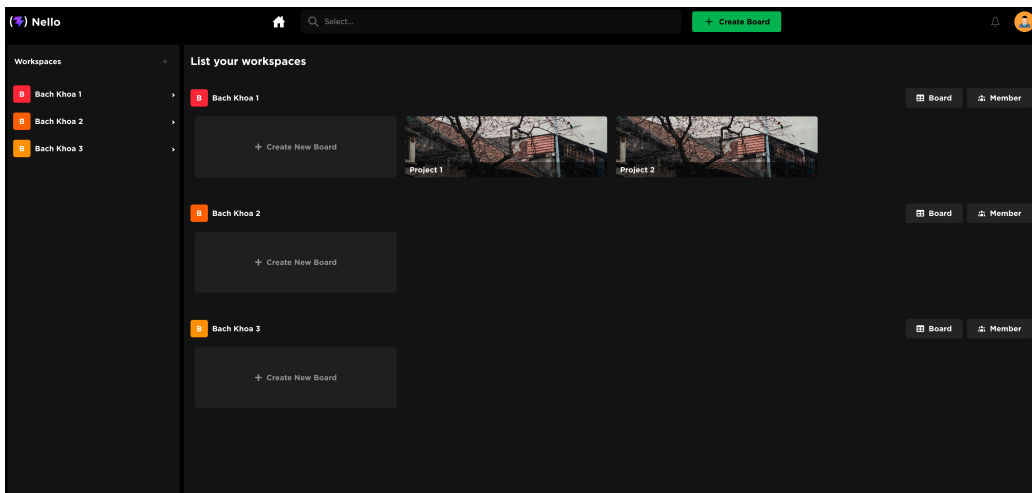
Hình 9: Màn hình Xác nhận OTP

3. **Màn hình Đăng nhập (/sign-in):** Form email và mật khẩu; thông báo lỗi khi thông tin không đúng; lưu token vào HttpOnly cookie; chuyển hướng đến Dashboard.



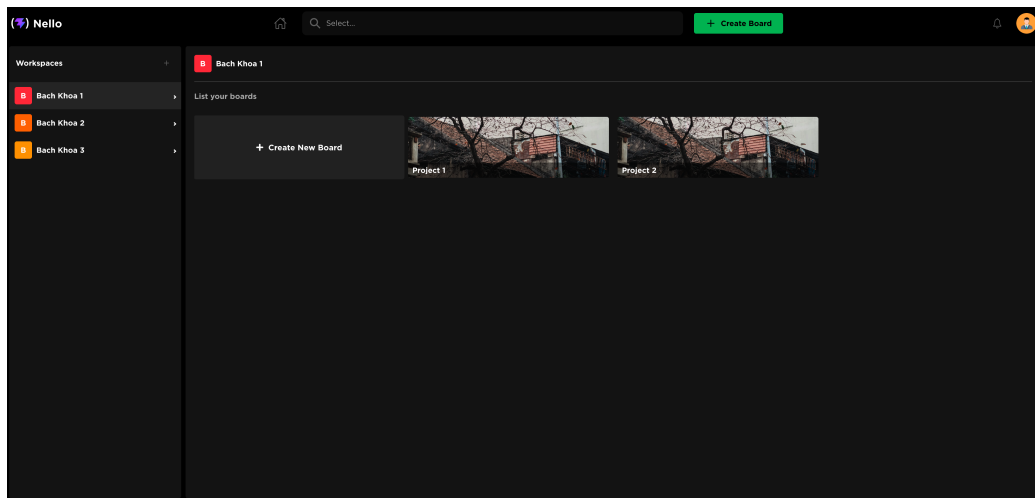
Hình 10: Màn hình Đăng nhập

4. **Dashboard:** Hiển thị danh sách Workspace trong sidebar; hiển thị Board theo từng Workspace dưới dạng thumbnail với ảnh nền; xử lý trạng thái trống khi chưa có dữ liệu; tự động làm mới token khi phiên hết hạn.



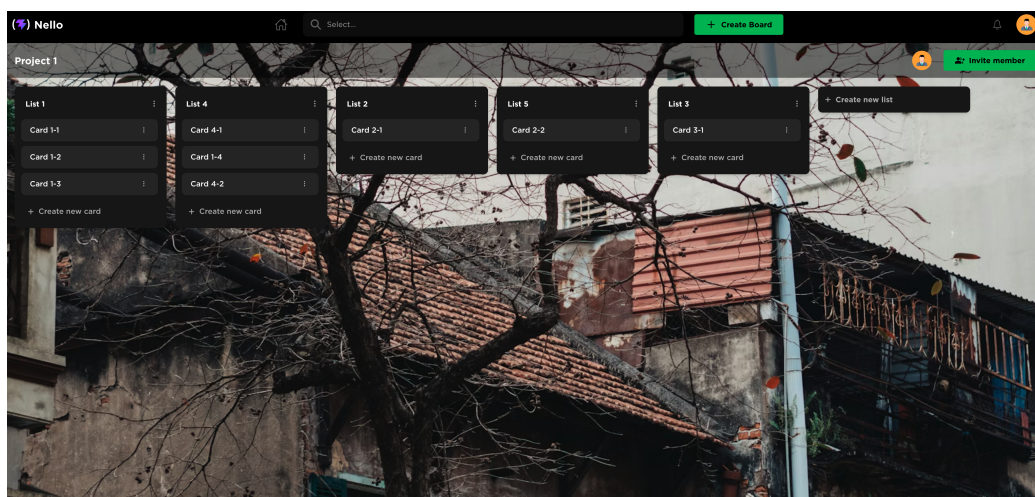
Hình 11: Màn hình Dashboard

5. **Màn hình Chi tiết Workspace (/workspaces/:id):** Thông tin Workspace, danh sách Board, danh sách thành viên với vai trò; modal mời thành viên bằng email.



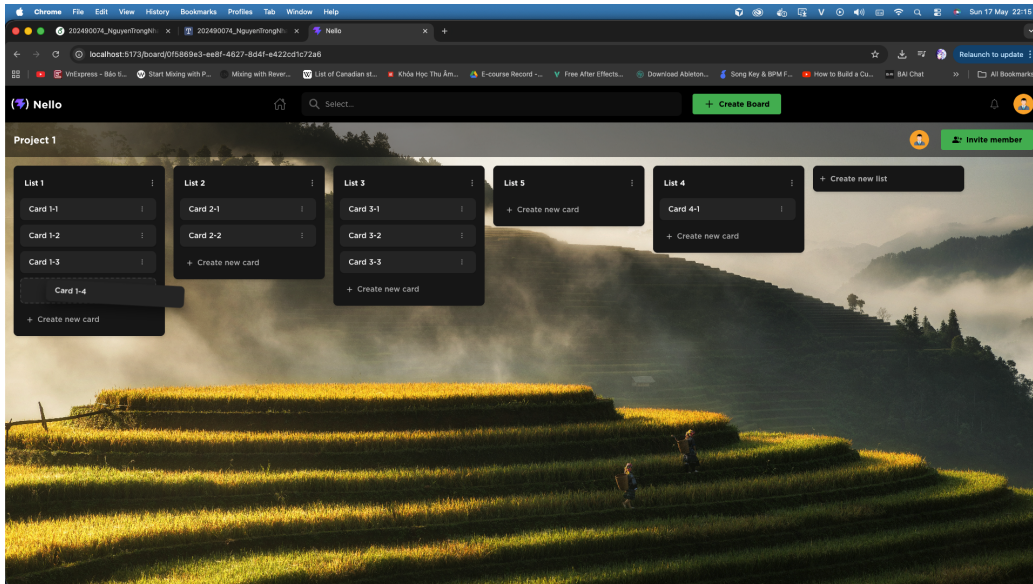
Hình 12: Màn hình Chi tiết Workspace

6. **Màn hình Board Kanban (/boards/:id):** Giao diện Kanban với các List theo dạng cột ngang; hiển thị Card trong từng List; form inline tạo List và Card; modal chi tiết Card; tính năng kéo thả List và Card; quản lý thành viên Board; badge cảnh báo Due Date; avatar thành viên trên Card.

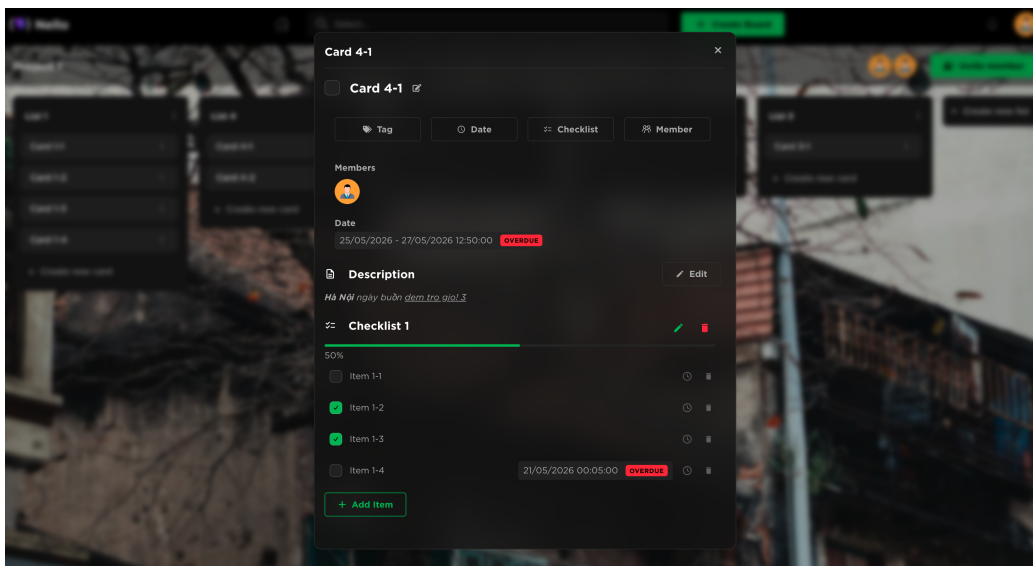


Hình 13: Màn hình Board Kanban

4.3 Hình ảnh minh họa



Hình 14: Tính năng kéo thả Card giữa các List



Hình 15: Modal Chi tiết Card với Checklist, Due Date và thành viên

4.4 Đánh giá kết quả

4.4.1 Ưu điểm

Qua quá trình phát triển và kiểm thử, hệ thống quản lý công việc Nello đạt được một số ưu điểm nổi bật:

- **Kiến trúc phân tách rõ ràng:** Frontend (ReactJS) và Backend (Spring Boot) hoàn toàn độc lập, tạo điều kiện bảo trì và mở rộng từng thành phần một cách linh hoạt.
- **Bảo mật theo tiêu chuẩn:** Toàn bộ xác thực được ủy thác cho Amazon Cognito. Token lưu trong HttpOnly cookie giúp bảo vệ khỏi tấn công XSS; tài nguyên Backend và cơ sở dữ liệu được cô lập trong mạng nội bộ VPC.
- **Trải nghiệm người dùng trực quan:** Giao diện Kanban quen thuộc với hỗ trợ kéo thả và cơ chế optimistic update cho phép người dùng thấy phản hồi tức thì mà không cần chờ phản hồi từ Backend.
- **Codebase chất lượng:** Áp dụng DTO và validation, xử lý ngoại lệ tập trung (Exception Handler), định dạng response chuẩn hóa trên toàn bộ API.

4.4.2 Hạn chế

Bên cạnh các kết quả đạt được, hệ thống còn tồn tại một số hạn chế cần được khắc phục trong giai đoạn tiếp theo:

- **Một số chức năng chưa hoàn thiện:** Các API Backend cho phép cập nhật và xóa Workspace (FR-08, FR-30) và cập nhật Board (FR-12) đã được triển khai đầy đủ; tuy nhiên, giao diện Frontend tương ứng chưa được xây dựng. Người dùng hiện không thể thực hiện các thao tác này qua giao diện web mà chỉ có thể gọi API trực tiếp.
- **Kiểm thử chưa bao phủ đầy đủ các trường hợp biên và luồng lỗi:** Phần lớn các test case được thực hiện tập trung vào luồng thành công (happy case) và một số lỗi cơ bản. Các kịch bản lỗi nâng cao như dữ liệu vượt giới hạn, truy cập đồng thời, mất kết nối hoặc dữ liệu không nhất quán chưa được kiểm thử, dẫn đến nguy cơ tồn tại lỗi tiềm ẩn trong các điều kiện sử dụng thực tế.

- **Lỗi kéo thả ở vị trí đầu/cuối:** Thuật toán tính `position` dựa trên `beforeId/afterId` chưa xử lý đầy đủ các trường hợp biên khi kéo Card hoặc List về vị trí đầu tiên hoặc cuối cùng, dẫn đến sai lệch thứ tự hiển thị sau một số thao tác liên tiếp.
- **Phân quyền VIEWER chưa hoàn thiện:** Vai trò VIEWER được định nghĩa trong hệ thống nhưng chưa được áp dụng đầy đủ ở cả tầng Backend (kiểm tra quyền tại API endpoint) và tầng Frontend (ẩn/vô hiệu hóa các thao tác chỉnh sửa).
- **Chưa triển khai lên môi trường Staging:** Hệ thống hiện chỉ hoạt động trên môi trường phát triển cục bộ. Nguyên nhân chính là do xung đột cấu hình `HttpOnly` cookie khi Frontend và Backend được triển khai trên các domain khác nhau, đòi hỏi phải điều chỉnh chính sách `SameSite` và `CORS`.
- **Chưa có tính năng đồng bộ thời gian thực:** Hệ thống hoạt động theo mô hình request-response thuần túy. Khi một thành viên thực hiện thay đổi, các thành viên khác đang xem cùng Board cần tải lại trang thủ công để cập nhật dữ liệu, ảnh hưởng đến trải nghiệm cộng tác.

5 CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

5.1.1 Kết quả đạt được

Sau 8 tuần thực hiện (từ 30/03/2026 đến 24/05/2026), hệ thống quản lý công việc Nello đã được hoàn thiện với các số liệu triển khai cụ thể như sau:

Hạng mục	Kết quả
Thời gian thực hiện	8 tuần (30/03/2026 – 24/05/2026)
Số chức năng hoàn thành	27/31 chức năng (87,1%)
Số API endpoint hoàn thành	43 endpoint trên 14 nhóm module
Số màn hình Frontend	6 màn hình chính
Số test case đã kiểm thử	Hơn 30 test case (tỷ lệ pass 90%)
Trạng thái Backend	Hoàn chỉnh
Trạng thái Frontend	Hoàn chỉnh các chức năng cốt lõi
Môi trường triển khai	Local development

Bảng 29: Tổng hợp kết quả học phần Project I

Đối chiếu với các mục tiêu đề ra ban đầu, các kết quả cụ thể đạt được như sau:

- **Xây dựng nền tảng quản lý khoa học:** Hoàn thành. Hệ thống hỗ trợ đầy đủ việc tổ chức công việc theo cấu trúc phân cấp Workspace → Board → List → Card, đáp ứng nhu cầu quản lý dự án đa cấp.
- **Quản lý thẻ công việc chi tiết:** Hoàn thành. Card hỗ trợ đầy đủ các thuộc tính gồm tiêu đề, mô tả, ngày bắt đầu, ngày đến hạn, gán thành viên và danh sách kiểm tra (Checklist) với thanh tiến độ trực quan.
- **Tích hợp cơ chế tương tác trực quan:** Hoàn thành. Tính năng kéo thả hoạt động ổn định cho cả List và Card trong Board, với phản hồi giao diện tức thì thông qua cơ chế optimistic update.
- **Đồng bộ dữ liệu và cộng tác:** Hoàn thành một phần. Các cập nhật về trạng thái công việc, phân công nhân sự và thời hạn được đồng bộ hóa với Backend sau mỗi thao tác; tuy nhiên, đồng bộ thời gian thực giữa các thành viên chưa được triển khai.

Hệ thống đã chứng minh tính khả thi của việc xây dựng một ứng dụng quản lý công việc theo mô hình Kanban với kiến trúc phân tách Frontend/Backend, tích hợp dịch vụ đám mây AWS và đảm bảo các tiêu chuẩn bảo mật cơ bản.

5.2 Các vấn đề còn tồn tại

5.2.1 Chức năng chưa hoàn thiện

Một số chức năng đã được thiết kế và triển khai ở tầng Backend nhưng giao diện Frontend tương ứng chưa được xây dựng:

- Cập nhật và xóa Workspace (**FR-08**, **FR-30**)
- Cập nhật và xóa Board (**FR-12**, **FR-31**)

Ngoài ra, một số tính năng đã được xác định là cần thiết nhưng nằm ngoài phạm vi thực hiện ban đầu:

Tính năng	Mô tả
Quản lý nhãn dán (Tag)	Tạo, gắn, tìm kiếm và lọc Card theo Tag màu sắc.
Hệ thống thông báo	Thông báo khi Card được giao, khi gần đến hạn hoặc khi được mời vào Workspace/Board.
Nhật ký hoạt động	Ghi nhận lịch sử tạo, cập nhật, xóa dữ liệu; hiển thị timeline hoạt động trong Card.
Board View	Xem Board theo nhiều mode view khác nhau.
Upload tệp đính kèm	Đính kèm tệp tin vào Card thông qua Amazon S3.
Đồng bộ thời gian thực	Cập nhật tự động giao diện khi thành viên khác thay đổi dữ liệu trong cùng workspace.

Bảng 30: Các tính năng chưa được triển khai

5.2.2 Hạn chế kỹ thuật

- **Lỗi kéo thả ở vị trí đầu/cuối danh sách:** Thuật toán tính `position` dựa trên `beforeId/afterId` chưa xử lý đầy đủ các trường hợp biên khi kéo Card hoặc List về vị trí đầu tiên (không tồn tại `beforeId`) hoặc vị trí cuối cùng (không tồn tại `afterId`), dẫn đến sai lệch thứ tự hiển thị sau một số thao tác kéo thả liên tiếp.

- **Phân quyền VIEWER chưa hoàn thiện:** Vai trò VIEWER được định nghĩa trong cơ sở dữ liệu nhưng chưa được kiểm soát đầy đủ ở tầng Backend (thiếu middleware kiểm tra quyền trên các write operation) và tầng Frontend (chưa ẩn hoặc vô hiệu hóa các nút thao tác chỉnh sửa tương ứng).
- **Chưa triển khai lên môi trường Staging:** Hệ thống hiện chỉ vận hành trên môi trường phát triển cục bộ. Vấn đề kỹ thuật cốt lõi là xung đột cấu hình HttpOnly cookie khi Frontend và Backend được đặt trên các domain khác nhau, đòi hỏi phải điều chỉnh chính sách SameSite và cấu hình CORS phù hợp với môi trường production.
- **Chưa có đồng bộ thời gian thực:** Hệ thống hoạt động theo mô hình request-response thuần túy. Khi một thành viên thực hiện thay đổi, các thành viên khác đang xem cùng Board cần tải lại trang thủ công để nhận cập nhật mới nhất.
- **Kiểm thử chưa bao phủ đầy đủ:** Phần lớn test case tập trung vào luồng thành công và một số lỗi cơ bản. Các kịch bản nâng cao như dữ liệu vượt giới hạn, truy cập đồng thời, mất kết nối mạng và dữ liệu không nhất quán chưa được kiểm thử, tiềm ẩn nguy cơ lỗi trong môi trường thực tế.

5.3 Phương hướng phát triển

5.3.1 Ưu tiên cao — Hoàn thiện hệ thống hiện tại

- **Sửa lỗi thuật toán kéo thả:** Rà soát và hoàn thiện thuật toán tính beforeId/afterId, đảm bảo xử lý đúng tất cả các trường hợp biên bao gồm kéo vào đầu danh sách, cuối danh sách và danh sách trống; đồng thời đảm bảo tính nhất quán giữa trạng thái Frontend và dữ liệu Backend.
- **Hoàn thiện phân quyền VIEWER:** Bổ sung kiểm tra vai trò tại các API endpoint write operation ở tầng Backend và ẩn hoặc vô hiệu hóa các thành phần giao diện thao tác tương ứng khi người dùng có vai trò VIEWER.
- **Giải quyết vấn đề triển khai Staging:** Lựa chọn và triển khai chiến lược quản lý token phù hợp với môi trường cross-domain, bao gồm cấu hình SameSite=None; Secure cho cookie hoặc chuyển sang cơ chế lưu token trong Authorization header, sau đó tiến hành triển khai hệ thống lên môi trường Staging trên hạ tầng AWS đã thiết kế.

- **Hoàn thiện giao diện cập nhật và xóa Workspace/Board:** Xây dựng các màn hình và modal giao diện tương ứng cho các API FR-08, FR-12, FR-30, FR-31 đã có sẵn ở Backend.

5.3.2 Ưu tiên trung bình — Mở rộng tính năng

- **Đồng bộ thời gian thực (Realtime Websocket):** Tích hợp Spring WebSocket với giao thức STOMP vào Backend và SockJS/STOMP client vào Frontend để đẩy sự kiện cập nhật dữ liệu (tạo, cập nhật, xóa Card; thay đổi Checklist; kéo thả) đến toàn bộ thành viên đang hoạt động trên cùng Board, nâng cao đáng kể trải nghiệm cộng tác nhóm.
- **Quản lý nhãn dán (Tag):** Triển khai tính năng tạo, chỉnh sửa và xóa Tag với màu sắc tùy chọn; gắn nhiều Tag cho một Card; tìm kiếm và lọc Card theo Tag trong Board.
- **Hệ thống thông báo (Notification):** Xây dựng cơ chế thông báo trong ứng dụng khi Card được giao cho người dùng, khi công việc sắp đến hạn hoặc đã quá hạn, và khi được mời vào Workspace hoặc Board.

5.3.3 Ưu tiên thấp — Phát triển dài hạn

- **Nhật ký hoạt động (Activity Log):** Ghi nhận lịch sử toàn bộ thao tác tạo, cập nhật và xóa dữ liệu; hiển thị timeline hoạt động theo thời gian thực trong modal Card Detail và màn hình Board.
- **Các chế độ xem thay thế:** Phát triển Calendar View hiển thị Card theo due date trên lịch tháng và Timeline View trình bày tiến trình công việc theo dòng thời gian, giúp người dùng có thêm góc nhìn tổng quan về tiến độ dự án.
- **Upload tệp đính kèm:** Tích hợp khả năng đính kèm tệp tin vào Card thông qua Amazon S3, hỗ trợ xem trước và tải xuống trực tiếp từ giao diện Card Detail.

Tài liệu

- [1] Amazon Web Services. *Amazon Cognito Developer Guide*. 2024. Truy cập tại: <https://docs.aws.amazon.com/cognito/latest/developerguide>. Ngày truy cập: 30/05/2026.
- [2] Amazon Web Services. *Amazon EC2 User Guide for Linux Instances*. 2024. Truy cập tại: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide>. Ngày truy cập: 30/05/2026.
- [3] Amazon Web Services. *Amazon RDS User Guide*. 2024. Truy cập tại: <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide>. Ngày truy cập: 30/05/2026.
- [4] Amazon Web Services. *Amazon S3 User Guide*. 2024. Truy cập tại: <https://docs.aws.amazon.com/AmazonS3/latest/userguide>. Ngày truy cập: 30/05/2026.
- [5] Amazon Web Services. *Amazon CloudFront Developer Guide*. 2024. Truy cập tại: <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide>. Ngày truy cập: 30/05/2026.
- [6] Amazon Web Services. *User Guide for Application Load Balancers*. 2024. Truy cập tại: <https://docs.aws.amazon.com/elasticloadbalancing/latest/application>. Ngày truy cập: 30/05/2026.
- [7] Atlassian. *Trello — Manage Your Team’s Projects From Anywhere*. 2024. Truy cập tại: <https://trello.com>. Ngày truy cập: 30/05/2026.
- [8] Atlassian. *Jira Software — Plan, Track, and Release Great Software*. 2024. Truy cập tại: <https://www.atlassian.com/software/jira>. Ngày truy cập: 30/05/2026.

A PHỤ LỤC A: DANH SÁCH API END-POINT ĐẦY ĐỦ

Bảng dưới đây liệt kê toàn bộ API endpoint đã được triển khai, gồm 14 nhóm chức năng với tổng số hơn 40 endpoint. Tất cả endpoint đều có tiền tố `/api/v1`.

Module	Method	Endpoint	Mô tả
Authentication	POST	<code>/auth/sign-up</code>	Đăng ký tài khoản mới
Authentication	POST	<code>/auth/confirm-sign-up</code>	Xác nhận OTP kích hoạt tài khoản
Authentication	POST	<code>/auth/sign-in</code>	Đăng nhập, nhận JWT qua HttpOnly Cookie
Authentication	POST	<code>/auth/refresh-token</code>	Làm mới access token
Authentication	POST	<code>/auth/resent-code</code>	Gửi lại mã OTP
User	GET	<code>/users</code>	Tìm kiếm user theo email hoặc tên
User	GET	<code>/users/me</code>	Lấy thông tin user hiện tại từ token
Workspace	POST	<code>/workspaces</code>	Tạo workspace mới
Workspace	GET	<code>/workspaces</code>	Lấy danh sách workspace của user
Workspace	GET	<code>/workspaces/:id</code>	Lấy chi tiết workspace
Workspace	GET	<code>/workspaces/:id/boards</code>	Lấy danh sách board thuộc workspace
Workspace	GET	<code>/workspaces/:id/members</code>	Lấy danh sách thành viên workspace
Workspace Member	POST	<code>/workspaces-members/:id</code>	Mời thành viên vào workspace
Board	POST	<code>/boards</code>	Tạo board trong workspace
Board	GET	<code>/boards/:id</code>	Lấy chi tiết board kèm lists và cards
Board	PUT	<code>/boards/:id</code>	Cập nhật tên board
Board	DELETE	<code>/boards/:id</code>	Xóa board
Board Member	POST	<code>/boards-members</code>	Thêm thành viên vào board

(tiếp theo trang sau)

Module	Method	Endpoint	Mô tả
Board Member	GET	/board-members	Tìm kiếm thành viên trong board
List	POST	/lists	Tạo list trong board
List	PATCH	/lists/:id	Cập nhật tên hoặc vị trí list
List	DELETE	/lists/:id	Xóa list (cascade xóa cards)
List	GET	/lists?boardId=	Lấy danh sách list theo thứ tự vị trí
Card	POST	/cards	Tạo card trong list
Card	GET	/cards/:id	Lấy chi tiết card
Card	PATCH	/cards/:id	Cập nhật thông tin hoặc di chuyển card
Card	DELETE	/cards/:id	Xóa card
Card	GET	/cards?boardId=	Lấy danh sách card theo board
Checklist	POST	/checklists	Tạo checklist trong card
Checklist	PATCH	/checklists/:id	Cập nhật tên hoặc vị trí checklist
Checklist	DELETE	/checklists/:id	Xóa checklist (cascade xóa items)
Checklist	GET	/checklists?cardId=	Lấy danh sách checklist kèm items
Checklist Item	POST	/checklist-items	Tạo item trong checklist
Checklist Item	PATCH	/checklist-items/:id	Cập nhật hoặc đánh dấu hoàn thành
Checklist Item	DELETE	/checklist-items/:id	Xóa item
Card Member	POST	/card-members	Gán thành viên vào card
Card Member	DELETE	/card-members/:id	Hủy gán thành viên khỏi card
Image	GET	/images	Lấy danh sách 8 ảnh nền board
WorkspaceCategory	GET	/workspace-categories	Lấy 8 danh mục workspace
Role	GET	/roles	Lấy danh sách vai trò (ADMIN, MEMBER, VIEWER)

Bảng 31: Danh sách API endpoint đầy đủ của hệ thống

B PHỤ LỤC B: CHI TIẾT BẢNG CƠ SỞ DỮ LIỆU

Hệ thống sử dụng PostgreSQL với 13 bảng dữ liệu, quản lý migration bằng Flyway (`V1__init.sql`). Khóa chính của các bảng nghiệp vụ sử dụng kiểu UUID; các bảng tham chiếu dùng INT.

Bảng roles

Cột	Kiểu dữ liệu	Ràng buộc
id	INT	PRIMARY KEY
name	role_name_enum	UNIQUE NOT NULL — ADMIN, MEMBER, VIEWER

Bảng images

Cột	Kiểu dữ liệu	Ràng buộc
id	INT	PRIMARY KEY
key	TEXT	NOT NULL — đường dẫn ảnh nền board

Bảng workspace_categories

Cột	Kiểu dữ liệu	Ràng buộc
id	INT	PRIMARY KEY
name	VARCHAR(255)	NOT NULL

Bảng users

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
cognito_id	VARCHAR(255)	NOT NULL, UNIQUE
full_name	VARCHAR(255)	NOT NULL
email	VARCHAR(255)	NOT NULL, UNIQUE
avatar_url	VARCHAR(255)	nullable
bio	TEXT	nullable
phone	VARCHAR(255)	nullable
address	VARCHAR(255)	nullable
deleted_at	TIMESTAMP	Soft delete
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Bảng workspaces

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
name	VARCHAR(255)	NOT NULL
description	VARCHAR(255)	nullable
category_id	INT	FK → workspace_categories(id)
created_by	UUID	FK → users(id) ON DELETE SET NULL
updated_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Bảng workspace_members

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
user_id	UUID	FK → users(id) ON DELETE CASCADE
workspace_id	UUID	FK → workspaces(id) ON DELETE CASCADE
role_id	INT	FK → roles(id) ON DELETE CASCADE
created_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
UNIQUE (user_id, workspace_id)		

Bảng boards

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
name	VARCHAR(255)	NOT NULL
background_url	TEXT	nullable
workspace_id	UUID	FK → workspaces(id) ON DELETE CASCADE
created_by	UUID	FK → users(id) ON DELETE SET NULL
updated_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Bảng board_members

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
user_id	UUID	FK → users(id) ON DELETE CASCADE
board_id	UUID	FK → boards(id) ON DELETE CASCADE
role_id	INT	FK → roles(id) ON DELETE CASCADE
created_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
UNIQUE (user_id, board_id)		

Bảng lists

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
name	VARCHAR(255)	NOT NULL
board_id	UUID	FK → boards(id) ON DELETE CASCADE
position	TEXT	NOT NULL — fractional indexing
created_by	UUID	FK → users(id) ON DELETE SET NULL
updated_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Bảng cards

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
title	VARCHAR(255)	NOT NULL
description	TEXT	nullable
start_date	TIMESTAMP	nullable
due_date	TIMESTAMP	nullable
is_completed	BOOLEAN	DEFAULT FALSE
position	TEXT	NOT NULL — fractional indexing
list_id	UUID	FK → lists(id) ON DELETE CASCADE
created_by	UUID	FK → users(id) ON DELETE SET NULL
updated_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Bảng card_members

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
user_id	UUID	FK → users(id) ON DELETE CASCADE
card_id	UUID	FK → cards(id) ON DELETE CASCADE
created_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
UNIQUE (user_id, card_id)		

Bảng checklists

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
name	VARCHAR(255)	NOT NULL
card_id	UUID	FK → cards(id) ON DELETE CASCADE
position	TEXT	NOT NULL
created_by	UUID	FK → users(id) ON DELETE SET NULL
updated_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Bảng checklist_items

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
name	VARCHAR(255)	NOT NULL
checklist_id	UUID	FK → checklists(id) ON DELETE CASCADE
position	TEXT	NOT NULL
is_completed	BOOLEAN	NOT NULL DEFAULT FALSE
due_date	TIMESTAMP	nullable
created_by	UUID	FK → users(id) ON DELETE SET NULL
updated_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

C PHỤ LỤC C: DANH SÁCH KỊCH BẢN KIỂM THỬ

Hệ thống đã được kiểm thử qua 5 đợt, bao phủ tổng cộng hơn 50 kịch bản kiểm thử. Toàn bộ kịch bản đều đạt kết quả Pass tại thời điểm kết thúc học phần.

Module Authentication (W3) — 14 kịch bản

Mã TC	Kịch bản kiểm thử	Kết quả
TC01	Đăng ký tài khoản với thông tin hợp lệ	Pass
TC02	Đăng ký với email đã tồn tại trong hệ thống	Pass
TC03	Đăng ký với thiếu trường bắt buộc	Pass
TC04	Xác nhận OTP với mã đúng	Pass
TC05	Xác nhận OTP với mã sai	Pass
TC06	Đăng nhập với thông tin hợp lệ	Pass
TC07	Đăng nhập với mật khẩu sai	Pass
TC08	Đăng nhập với tài khoản chưa xác nhận email	Pass
TC09	Kiểm tra token được lưu vào HttpOnly Cookie	Pass
TC10	Làm mới access token với refresh token hợp lệ	Pass
TC11	Làm mới token khi thiếu refresh token	Pass
TC12	Làm mới token khi token đã hết hạn	Pass
TC13	Gửi lại mã OTP cho user hợp lệ	Pass
TC14	Gửi lại mã OTP cho user không tồn tại	Pass

Module Workspace & Board (W4) — 16 kịch bản

Mã TC	Kịch bản kiểm thử	Kết quả
TC01	Tạo workspace với thông tin hợp lệ	Pass
TC02	Tạo workspace với tên trống	Pass
TC03	Tạo workspace với token xác thực sai	Pass
TC04	Lấy danh sách workspace của user hiện tại	Pass
TC05	Lấy danh sách workspace khi chưa có workspace nào	Pass
TC06	Tạo board trong workspace hợp lệ	Pass
TC07	Tạo board trong workspace không tồn tại	Pass
TC08	Tạo board khi user không có quyền trong workspace	Pass
TC09	Cập nhật tên board	Pass
TC10	Cập nhật board không tồn tại	Pass
TC11	Xóa board	Pass
TC12	Xóa board khi không có quyền	Pass
TC13	Tạo list trong board hợp lệ	Pass
TC14	Tạo list trong board không tồn tại	Pass
TC15	Cập nhật tên list	Pass
TC16	Lấy danh sách list theo board	Pass

Module Card & Checklist (W5) — 9 kịch bản

Mã TC	Kịch bản kiểm thử	Kết quả
TC17	Tạo card trong list hợp lệ	Pass
TC18	Lấy chi tiết card	Pass
TC19	Cập nhật thông tin card (title, description, due-Date)	Pass
TC20	Di chuyển card giữa các List (drag-and-drop)	Pass
TC21	Xóa card	Pass
TC22	Lấy danh sách card theo board	Pass
TC23	Tạo checklist trong card	Pass
TC24	Tạo checklist item trong checklist	Pass
TC25	Cập nhật trạng thái hoàn thành của checklist item	Pass

Frontend Integration (W6) — 9 kịch bản

Mã TC	Kịch bản kiểm thử	Kết quả
TC01	Luồng xác thực đầy đủ: Đăng ký → Xác nhận OTP → Đăng nhập	Pass
TC02	Thông báo lỗi khi đăng ký với user đã tồn tại	Pass
TC03	Thông báo lỗi khi đăng nhập với mật khẩu sai	Pass
TC04	Thông báo lỗi khi xác nhận OTP sai	Pass
TC05	Tải màn hình Dashboard và hiển thị danh sách workspace	Pass
TC06	Tạo workspace mới từ Dashboard	Pass
TC07	Tạo board mới trong workspace	Pass
TC08	Xử lý empty state khi chưa có workspace	Pass
TC09	Auto refresh token khi access token hết hạn	Pass

Board UI (W7) — 8 kịch bản

Mã TC	Kịch bản kiểm thử	Kết quả
TC01	Tải màn hình Workspace Detail và hiển thị danh sách board và thành viên	Pass
TC02	Mời thành viên vào workspace qua modal	Pass
TC03	Tải màn hình Board Kanban với đầy đủ List và Card	Pass
TC04	Tạo List mới trong Board	Pass
TC05	Tạo Card mới trong List	Pass
TC06	Xóa Card với modal xác nhận	Pass
TC07	Xóa List với modal xác nhận	Pass
TC08	Mở Card Detail Modal và hiển thị Checklist, Due Date, Members	Pass